

Resúmenes — Criptografía y Seguridad (72.44)

Primera Parte: Criptografía — Primer Parcial
ITBA

Material de estudio basado en clases, transcripts y parciales

Índice

1. Introducción a la Criptografía	3
1.1. Qué es la criptografía	3
1.2. Criptosistema (visión informal)	3
1.3. Principio de Kerckhoffs	4
1.4. Seguridad (noción informal)	5
1.5. Breve historia de la criptografía	5
1.6. Cifrado por rotación (ROT-X / César)	5
1.7. Cifrado por sustitución (monoalfabético)	6
1.8. Cifrado de Vigenère (sustitución polialfabética)	8
1.9. Máquina Enigma (rotores)	9
1.10. Hacia la criptografía moderna	10
1.11. Secreto perfecto (introducción)	10
2. Cifrado	12
2.1. Cifrado simétrico (criptosistema)	12
2.2. Secreto perfecto (Shannon)	12
2.2.1. Teorema (condición necesaria): $\#K \geq \#M$	13
2.2.2. Método para demostrar secreto perfecto (PASO A PASO)	13
2.2.3. Ejemplo trabajado: clave NO aleatoria (rompe el secreto perfecto)	14
2.3. One-Time Pad (OTP) / Vernam	14
2.3.1. El OTP tiene secreto perfecto (demostración)	15
2.3.2. Limitaciones del OTP (las malas noticias)	16
2.4. Seguridad computacional	16
2.4.1. Pruebas de seguridad (experimentos)	16
2.4.2. Propiedades CPA y necesidad de cifrado probabilístico	17
2.5. Cifrado de flujo	18
2.6. Cifrado de bloque	19
2.6.1. DES, 3-DES, AES	19
2.7. Modos de encadenamiento	20
2.7.1. ECB (Electronic Codebook)	20
2.7.2. CBC (Cipher Block Chaining)	21
2.7.3. Otros modos: CFB, OFB, CTR	22
2.7.4. Propagación de errores en CBC (NÚCLEO de examen)	23
2.8. Estado de un criptosistema	24
3. MACs y Cifrado Autenticado	25
3.1. El nuevo escenario de ataque: integridad	25
3.2. Qué es un MAC	26
3.3. Seguridad de un MAC: infalsificabilidad (Mac-forge)	26
3.4. MÉTODO para romper un MAC inseguro	27

3.5.	Construcción de MAC: CBC-MAC	28
3.6.	Funciones de hash	29
3.7.	HMAC: MAC a partir de una función de hash	31
3.8.	Cifrado autenticado: confidencialidad + integridad	31
3.9.	No-repudio: por qué los MAC no lo dan	33
4.	Cifrado Asimétrico y Firma Digital	34
4.1.	El problema de distribución de claves	34
4.2.	La idea de la criptografía asimétrica	34
4.3.	Repaso de álgebra y aritmética	35
4.4.	Intercambio de claves	36
4.4.1.	Protocolo Diffie-Hellman	36
4.5.	Criptosistema asimétrico	37
4.5.1.	RSA (Rivest, Shamir, Adleman)	38
4.5.2.	ElGamal (basado en DH)	40
4.6.	Firmas digitales	40
4.6.1.	RSA-Signature y por qué se firma el HASH	41
4.7.	PKI y certificados digitales	42
5.	Protocolos	44
5.1.	Qué es un protocolo criptográfico	44
5.2.	Distribución de claves: KDC (simétrico) vs PKI (asimétrico)	44
5.2.1.	Análisis de protocolos KDC: Needham-Schroeder paso a paso	45
5.3.	Certificados y PKI	48
5.3.1.	Cadenas de firmas y CA raíz	49
5.4.	Canal seguro: SSL / TLS	50
5.4.1.	TLS Handshake	51
5.5.	Intercambio de claves: Diffie-Hellman y Shamir's no-key	52
5.6.	Secreto compartido de Shamir (umbrales / threshold)	53
5.6.1.	Método (paso a paso)	54
5.6.2.	Ejemplo completo resuelto $(t, n) = (3, 5)$ en $\mathbb{Z}_{11}$	55

1. Introducción a la Criptografía

Esta unidad usa la **historia de la criptografía** como vehículo para introducir, de forma intuitiva, los conceptos fundamentales: criptosistema, clave, principio de Kerckhoffs, criptoanálisis (ataque por fuerza bruta y análisis estadístico) y la noción de seguridad. Cierra con una primera formalización (terna de algoritmos) y la definición de **secreto perfecto** de Shannon, cuyo desarrollo formal continúa en la unidad 2.

“Hoy vamos a ver fundamentos, vamos a ver ideas. No vamos a entrar en el campo formal. Vamos a aprovechar la historia de la criptografía como para ir entendiendo los conceptos básicos.”

— dicho en clase

► En el parcial

Los cifrados históricos son el **vehículo recurrente** para discutir secreto perfecto. En distintos años se pidió analizar: Hill (2015), transposición por columnas (2016), cilindro de Jefferson (2017), homofónico (2018), Vigenère (2025-2). Tareas típicas: **calcular los tamaños de los espacios $\#M$, $\#K$, $\#C$ y demostrar o refutar secreto perfecto**. También caen: principio de Kerckhoffs (V/F), análisis de frecuencias e índice de coincidencia, y distinguir sustitución vs. transposición / confusión vs. difusión / linealidad.

1.1. Qué es la criptografía

Del griego: **escritura secreta**. Nace de la necesidad de ocultar información (“desde que el hombre es hombre y se agrupa, existe la necesidad de mantener alguna cosa en secreto”).

Definición: Criptografía

Conjunto de **funciones matemáticas y técnicas** que permiten asegurar distintas propiedades sobre la información. Son las herramientas básicas desde donde se construye la seguridad.

El concepto moderno trasciende la sola **confidencialidad** (esconder información) y abarca otras propiedades. Usos cotidianos donde hay criptografía “oculta en todos lados”: tráfico web e inalámbrico (Wi-Fi, satélite), protección de archivos en disco, autenticación de usuarios (segundo factor), protección de contenido (DRM, e-readers), firmas digitales, voto electrónico y dinero electrónico (Bitcoin / criptomonedas).

Existe una distinción de especialidades:

- **Criptografía**: estudio (diseño) de las funciones.
- **Criptoanálisis**: estudio de los **ataques** a esas funciones.

! Importante — remarcado en clase

Objetivo secundario de la materia: entender la **complejidad** que hay detrás de las funciones criptográficas para **no inventar primitivas propias**. El profe estima que hay unas 30 o 40 personas en el mundo capaces de construir funciones que sobrevivan al escrutinio público. Como ingenieros, debemos **usar las que ya existen**.

1.2. Criptosistema (visión informal)

La construcción más básica y común es el **criptosistema**: en principio, un conjunto de dos funciones.

- **Cifrado / encriptar**: toma un mensaje (texto plano) y lo transforma en algo aparentemente sin sentido (texto cifrado).

- **Descifrado / desenscriptar:** toma el mensaje cifrado y recupera el mensaje original.

Función de cifrado. Es una función de **dos parámetros**:

$$\text{Mensaje plano} \times \text{clave} \rightarrow \text{Mensaje cifrado}, \quad e_k(p) = c.$$

Diferentes representaciones, mismo significado:

$$e_k(p) = \text{enc}_k(p) = e(k, p) = \{p\}_k.$$

(La notación $\{p\}_k$ se usa más en protocolos: lo que está entre llaves está cifrado con la clave del subíndice.)

Función de descifrado. También de dos parámetros:

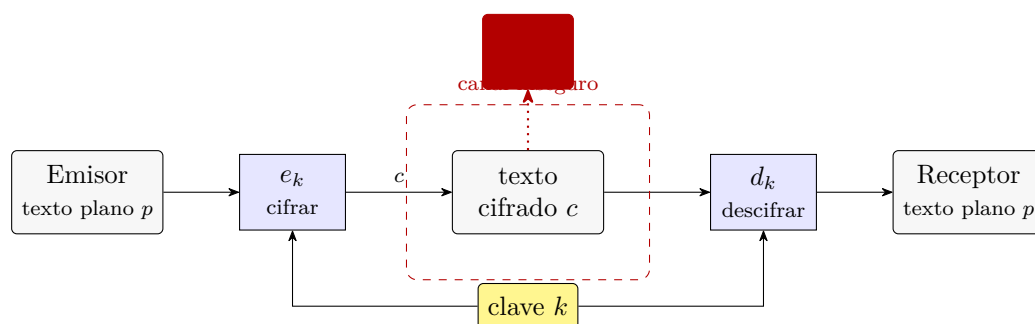
$$\text{Mensaje cifrado} \times \text{clave} \rightarrow \text{Mensaje plano}, \quad d_k(c) = p.$$

La clave. Es un **bloque arbitrario de información**. Se la separa del mensaje y se le da un nombre especial porque, al probar la seguridad de un sistema, **siempre se presupone que la clave está protegida**.

! Importante — remarcado en clase

Ejercicio mental fundamental en seguridad: **ponerse en el lugar del atacante**. La única constante es que, donde aparezca algo llamado *clave*, se asume que el atacante **no la tiene**; y si la consigue, “se acabó la seguridad”.

¿Para qué sirve? Resuelve el problema de **controlar el acceso y la diseminación** de la información. La idea: “patear el problema”. Proteger un mensaje de tamaño arbitrario es difícil; en cambio, ciframos el mensaje con una clave y entonces el texto cifrado puede almacenarse o transmitirse por cualquier lado (es público), mientras solo haya que **proteger la clave**, que es mucho más chica. En los criptosistemas modernos la clave **no depende del tamaño del mensaje**: se pueden cifrar terabytes con una clave de 128 bits.



Modelo de comunicación cifrada: el emisor cifra con e_k , el cifrado c viaja por un canal inseguro donde Eve puede espiar, y el receptor recupera p con d_k . La misma clave k entra a cifrado y descifrado; se asume que Eve no la tiene.

1.3. Principio de Kerckhoffs

Definición: Principio de Kerckhoffs

Un criptosistema debe ser **seguro incluso si todo sobre el sistema** (todo el algoritmo, todas las funciones), **excepto la clave**, es de público conocimiento.

Consecuencia para el análisis: la seguridad **no puede depender** de que el atacante desconozca el algoritmo o el código fuente. Hay que asumir que el atacante **conoce el criptosistema usado** y eventualmente el código; lo único secreto es la clave.

△ Cuidado — error típico

Error clásico de V/F. La afirmación “la seguridad solo depende de que el *algoritmo* esté protegido” es **FALSA** (Recup. 2025). Lo correcto: la seguridad depende **solo de la clave**, no de mantener secreto el algoritmo.

1.4. Seguridad (noción informal)

Primera intuición de un alumno: “es seguro si quien no tiene la clave no puede descifrar el mensaje en tiempo razonable”. El profe la refina: se pide **más**.

Definición: Seguridad (informal)

Un criptosistema es seguro si **ningún adversario puede computar cualquier función del texto plano** a partir del mensaje cifrado que posee. Significa que no se puede recuperar:

- el mensaje,
- parte del mensaje,
- ni siquiera el **sentido** del mensaje.

“No es solamente que no lo puedas decodificar: es mucho más fuerte, no se puede extraer ningún tipo de información a partir del texto cifrado si no se tiene la clave.”

— dicho en clase

Ejemplos de por qué importa “parte” y “sentido”: un documento gigante cuyo resumen está en la primera página; o una negociación secreta donde al adversario le basta saber si la respuesta será *positiva* o *negativa* (sin descifrar todo) para obtener una ventaja ilegal.

1.5. Breve historia de la criptografía

~2500 AC	Egipto: sustitución (jeroglíficos no estándar).
~700–300 AC	Escítala (Scitala): transposición con báculos.
50 AC	César: rotación (subtipo de sustitución).
800 DC	Primeros documentos de criptoanálisis: análisis de frecuencias.
~1500	Sustitución polialfabética (un símbolo cifrado representa distintos símbolos del original).
1553	Cifrado de Vigenère .
1939	Enigma / Bombe: ataques de fuerza bruta (protocomputadoras).
1949	Shannon: <i>Information Theory & Cryptography</i> , inicio de la criptografía moderna .

- **Escítala**: báculo de cierto tamaño/topología; se enrolla una tira de cuero y se escribe transversalmente. Para descifrar se necesita el **mismo báculo**. Es **transposición**.
- **César**: rotación **fija de 3** posiciones en el alfabeto romano. Documentado en campañas militares.
- Las dos grandes revoluciones: (1) los polialfabéticos (~400 años de dominio); (2) la criptografía moderna, contemporánea a la 2.^a Guerra Mundial (Shannon, teoría de la información).

1.6. Cifrado por rotación (ROT-X / César)

Se asigna un orden a los símbolos (alfabeto, comenzando por 0). La clave k es un número entre 1 y 26 (cantidad de letras -1). Cifrar = reemplazar cada letra por la que ocupa k posiciones más adelante, de forma **cíclica** (después de la **z** vuelve a la **a**).

$$e(\text{prueba}, 4) = \text{tv y ife}$$

Criptografía. Debilidad: hay pocas claves. Conociendo el algoritmo (Kerckhoffs) se prueban todas las claves por **fuerza bruta**:

$k = 1$	$d(1, \text{tvyife}) = \text{suxhed}$	sin sentido
$k = 2$	$d(2, \text{tvyife}) = \text{rtwgdc}$	sin sentido
$k = 3$	$d(3, \text{tvyife}) = \text{qsvfcb}$	sin sentido
$k = 4$	$d(4, \text{tvyife}) = \text{prueba}$	¡con sentido!

! Importante — remarcado en clase

El ataque por **fuerza bruta siempre está disponible** para el atacante, incluso contra el criptosistema más complejo: dado el texto cifrado y conocido el algoritmo, nada impide probar todas las claves. La debilidad de ROT-X no es que se pueda hacer fuerza bruta, sino que las **claves son pocas**.

△ Cuidado — error típico

La fuerza bruta tiene un **prerrequisito**: poder **discriminar un descifrado válido de uno inválido**. Si el mensaje es una sola letra elegida con distribución uniforme, el cifrado puede ser cualquier letra y **no se puede saber** cuál era el original (los descifrados son todos igual de “válidos”). Este es el germen de la idea de secreto perfecto.

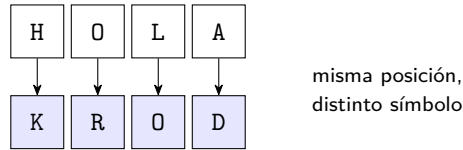
Redundancia y distancia de unicidad (Shannon). Los lenguajes naturales tienen mucha **redundancia** (mecanismo de control de errores: podemos leer una frase sin vocales). La **redundancia** es la diferencia entre el tamaño del mensaje y la información que realmente lleva (comprimir = recodificar eliminando redundancia; p. ej. Huffman). La **distancia de unicidad** es la longitud mínima que debe tener un mensaje para que la probabilidad de que exista más de un descifrado con sentido sea despreciable. Para lenguajes naturales (español, inglés) está en el orden de los **4 caracteres**: por encima de ese umbral, la fuerza bruta que encuentra un texto con sentido suele dar el resultado correcto.

1.7. Cifrado por sustitución (monoalfabético)

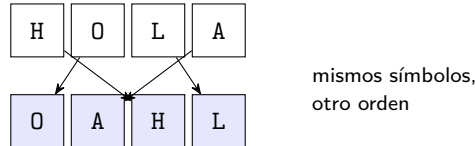
Para cada símbolo del alfabeto se establece una **transformación única** (una permutación del alfabeto): no puede haber dos símbolos distintos que se transformen en lo mismo (si no, no sería invertible). Reemplaza un símbolo por otro (que puede ser de otro alfabeto: letras, símbolos, números).

```
k = dublcmfthijnzpxqeaosvkrwgy
    abcdefghijklmnopqrstuvwxyz
esto es una prueba → cosx co vpd qavcud
```

Sustitución (reemplaza el símbolo)



Transposición (reordena posiciones)



Sustitución vs. transposición. La sustitución cambia cada símbolo por otro manteniendo su posición; la transposición conserva los mismos símbolos pero los reubica.

Tamaño del espacio de claves. La clave es la permutación del alfabeto. Con el alfabeto español de **27** símbolos:

$$\#K = 27! \approx \text{número de 28 dígitos (cuatrillones)}.$$

Es un número “bestialmente alto”: la fuerza bruta es inviable. **No** tiene la debilidad de ROT-X (pocas claves).

► En el parcial

“¿Cuántas claves tiene este criptosistema?” Para sustitución monoalfabética sobre un alfabeto de n símbolos: $\#K = n!$ (es una **permutación**). Para el español, $n = 27 \Rightarrow 27!$.

Criptografía: análisis de frecuencias.

△ Cuidado — error típico

Debilidad distinta: la sustitución **preserva las características estadísticas del lenguaje**. Una misma letra del plano siempre va al mismo símbolo cifrado, así que la letra que más aparece en el plano es la que más aparece en el cifrado.

Procedimiento (resoluble incluso por un chico, o sin conocer el idioma usando solo los indicadores estadísticos):

1. Obtener la **tabla de frecuencias** estimadas del lenguaje.
2. Calcular la frecuencia de cada símbolo en el texto cifrado.
3. Asumir que los símbolos más probables se corresponden (en español: E, A, S, ...).
4. Usar grupos de 2 y 3 letras comunes (*el, la, de, los, que...*), reglas (*q* siempre seguida de *u*; no hay 3 consonantes juntas), reemplazar y verificar sentido.

Tabla de frecuencias del español (%).

Frecuencia alta

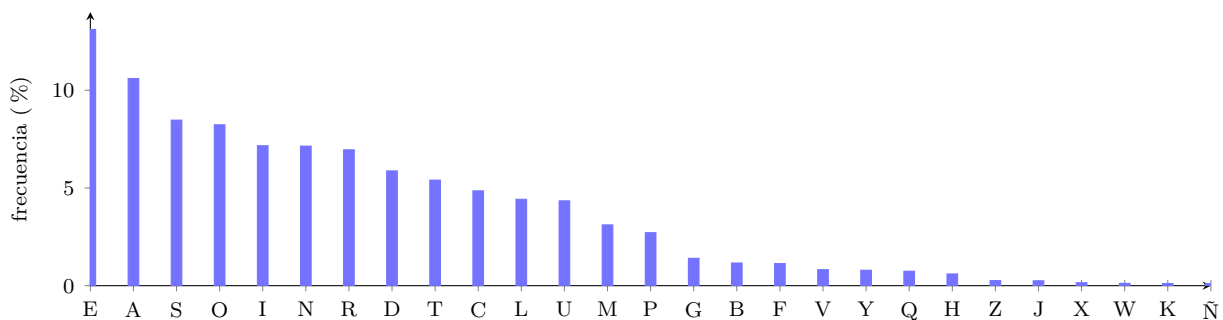
E 13,11 A 10,60 S 8,47
O 8,23 I 7,16 N 7,14
R 6,95 D 5,87 T 5,40

Frecuencia media

C 4,85 L 4,42 U 4,34
M 3,11 P 2,71 G 1,40
B 1,16 F 1,13 V 0,82

Frecuencia baja

Y 0,79 Q 0,74 H 0,60
Z 0,26 J 0,25 X 0,15
W 0,12 K 0,11 Ñ 0,10



Frecuencias de las letras del español (%). El perfil escalonado (E, A, S, O... dominantes) es lo que la sustitución monoalfabética no logra ocultar: el análisis de frecuencias compara este histograma con el del texto cifrado.

! Importante — remarcado en clase

La debilidad estadística **se traslada al mundo digital**: cifrar un MP4, un Word, etc., también deja estructura estadística (e incluso *headers* con valores fijos), atacable con los mismos mecanismos. Una cosa es decir que un criptosistema es **débil / fácil de romper** y otra es **efectivamente romperlo**: cuanto más complejo el sistema, más grande el espacio de búsqueda.

△ Cuidado — error típico

“Es seguro porque nadie lo rompió” es un criterio **malo**. Muchos criptosistemas fueron considerados buenos solo porque nadie les había dedicado el tiempo necesario, no por ser fundacionalmente seguros.

Gancho (crib / known-plaintext). Saber que en el plano aparece una secuencia conocida (p. ej. un saludo fijo al inicio, o “LACABEZA”) acelera mucho el ataque. Históricamente, los aliados aprovecharon que los mensajes alemanes empezaban casi siempre con el mismo saludo.

1.8. Cifrado de Vigenère (sustitución polialfabética)

Es una **composición de cifrados de rotación**: un mismo símbolo del plano puede transformarse en **distintos** símbolos cifrados. La clave es una secuencia de n números (o letras, con A= 0, B= 1, ...); cada elemento indica cuántas posiciones rotar la letra correspondiente. Al terminar la clave, se vuelve a empezar (el mensaje se divide en **bloques** del largo de la clave).


```
K = ECFD = 4253    (long 4)
esto es una prueba
4253 42 534 253425 342 ...
→ iuyrdgxcyofcttzhfc
```

! Importante — remarcado en clase

Vigenère **rompe las propiedades estadísticas** de la sustitución simple: la misma letra del plano puede dar letras cifradas distintas, y el mismo símbolo cifrado puede provenir de letras distintas. La letra que más aparece deja de ser el símbolo más frecuente del cifrado. Creado en **1553**, se lo consideró seguro casi **300 años**, hasta que en **1863 Friedrich Kasiski** publicó un método para resolverlo.

Criptanálisis: Test de Kasiski (dos fases). Fase 1 — **determinar la longitud de la clave (= tamaño de bloque)**. Aparecen **n-gramas repetidos** cuando los mismos símbolos del plano caen en la misma posición dentro del bloque (se rotan igual). Entonces:

1. Buscar secuencias repetidas en el texto cifrado.
2. Calcular la **distancia** (en caracteres) entre cada par de apariciones.
3. Esas distancias deben ser **múltiplos** del tamaño de bloque, así que la longitud de la clave es el **MCD** de las distancias halladas.

Ejemplo: GGMP ($\times 3$) separadas 256 y 104; YEDS ($\times 2$) separada 72; HASE ($\times 2$) separada 156; VSUE ($\times 2$) separada 32.

$$\text{MCD}(32, 72, 104, 156, 256) = 4.$$

Es una prueba estadística: puede fallar si dos secuencias coinciden por casualidad (el MCD da 1); en ese caso se van descartando muestras de a una hasta que el MCD vuelva a subir.

Fase 2 — resolver cada bloque (método de coincidencia mutua). Conocida la longitud (p. ej. 4), se agrupan las letras por posición: todas las 1.^a de cada bloque, todas las 2.^a, etc. Cada grupo está rotado por **una única** cantidad, así que su histograma de frecuencias se parece al del lenguaje. Idea de Kasiski:

- Al **juntar** dos grupos con rotaciones distintas, el histograma se rompe.
- Pero existe una rotación relativa que los **realinea**: rotando un grupo respecto del otro hasta que el histograma combinado se mantenga, se obtiene la **diferencia relativa de rotación** entre posiciones.
- Encadenando todos los grupos, el problema se reduce a un **único cifrado por rotación** (todo el mensaje desplazado por el mismo número), que se resuelve probando las ~ 26 posibilidades.

! Importante — remarcado en clase

A partir de Vigenère, los ataques dejan de ser genéricos: **ya no hay una solución única** para toda la familia polialfabética. El método de Kasiski **explota el conocer el algoritmo de Vigenère**; cada criptosistema polialfabético requiere un ataque *ad hoc*. (Para sustitución simple sí existía un algoritmo genérico.)

1.9. Máquina Enigma (rotores)

Estado del arte en la 2.^a Guerra Mundial: criptosistema polialfabético de **rotores** usado por Alemania. Cada rotor es como una sustitución simple (entra una letra, sale otra); al rotar, el mapeo cambia. Enigma original tenía **3 rotores** (hubo versiones de 4):

- Cada vez que se tipea una letra, rota un rotor; al dar la vuelta, rota el siguiente (como un cuentakilómetros).

- Período de repetición $\approx 26^3$, mucho mayor que cualquier mensaje: en la práctica, la secuencia **no se repetía**. Equivale a una sustitución simple que **varía letra a letra**, con “bloque” de 26^3 en vez de los 4 de Vigenère.
- Un **plugboard** (tablero) cruzaba conexiones; un reflector hacía que la señal “rebotara”, de modo que la **misma máquina** servía para cifrar y descifrar (poniendo la misma clave inicial).
- La **clave** era la disposición inicial de los rotores (cambiaba cada día según un cuaderno por oficial).

Los aliados (liderados en el criptoanálisis por **Turing**, entre otros) hicieron **ingeniería inversa** tras capturar una máquina y construyeron una computadora (Bombe) para fuerza bruta. No alcanzaba el cómputo; lo resolvieron usando **ganchos** (mensajes que empezaban con un saludo conocido) para reducir el espacio de búsqueda. **No rompieron Enigma de forma general**, solo ese uso; los submarinos con 4 rotores no fueron descifrados durante la guerra. De este esfuerzo surge prácticamente la computadora moderna (arquitectura de von Neumann).

1.10. Hacia la criptografía moderna

La 2.^a Guerra generó una **crisis**: no había criterio para dar fiabilidad de la seguridad. La definición era “es seguro porque nadie lo rompió”, y al juntar gente brillante se rompieron cosas que se creían seguras. De ahí nacen los **tres pilares** de la criptografía moderna:

1. **Representaciones formales** de los criptosistemas (ya no recetas algorítmicas).
2. Definiciones del **modelo de amenaza** (no hay una única definición de “seguro”; Enigma quizá era seguro, pero *como lo usaban* no lo era).
3. **Pruebas formales** de seguridad.

Los cifrados clásicos vistos **no se usan en la práctica** (son fáciles de romper algorítmicamente), pero son la **base** sobre la que se construye lo moderno.

Criptosistema (primera definición formal).

Definición: Criptosistema (terna de algoritmos)

- $\text{Gen} : () \rightarrow K$ — **generador de clave**. Suele ser sin parámetros; *determina el espacio de claves* (cuántas y cuáles son posibles).
- $\text{Enc} : K \times P \rightarrow C$ — **cifrado**.
- $\text{Dec} : K \times C \rightarrow P$ — **descifrado**.

Propiedad básica (corrección): para todo m y k válidos,

$$d_k(e_k(m)) = m.$$

Conjuntos: K (espacio de claves), P o M (espacio plano, todos los mensajes), C (espacio cifrado).

△ Cuidado — error típico

La propiedad básica es solo “sanitaria”: la **función identidad** (ignorar la clave y devolver el mensaje) también la cumple. Por eso esta definición **no** habla todavía de seguridad — porque no hay una única noción de seguridad.

1.11. Secreto perfecto (introducción)

Shannon define el primer criterio formal de seguridad: el **secreto perfecto**.

Definición: Secreto perfecto (Shannon)

Dado un criptosistema (Gen, e, d) , posee secreto perfecto si para **toda** distribución de probabilidades en M , cada mensaje m y cada texto cifrado c con $\Pr[C = c] > 0$:

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

Interpretación: la probabilidad que se conoce *a priori* sobre el mensaje **no se modifica** al conocer el texto cifrado. Conocer c no aporta **ninguna información** sobre m .

“Es mucho más fuerte que no poder decodificar el mensaje: no podés extraer nada de información. Un criptosistema con secreto perfecto es imposible de romper, incluso por fuerza bruta.”

— dicho en clase

Ejemplo (mensaje sí/no con $\Pr[\text{sí}] = 0,8$ a priori): si el sistema tiene secreto perfecto, decirte el cifrado no cambia ese 0,8 a 0,81 ni a 0,79.

△ Cuidado — error típico

Error típico de examen: confundir “secreto perfecto” con “no se puede romper / decodificar”. El criterio es la **definición formal de Shannon**, $\Pr[M = m \mid C = c] = \Pr[M = m]$, que es **más fuerte**: no se aprende *nada*. Como consecuencia (se demuestra en la unidad 2), exige

$$\#K \geq \#M.$$

! Importante — remarcado en clase

Paradoja del secreto perfecto: como $c = e_k(m)$, las variables aleatorias C y M están **relacionadas por una función** (son dependientes por diseño); sin embargo, la condición de secreto perfecto pide que se comporten como **estadísticamente independientes** ($\Pr[M = m \mid C = c] = \Pr[M = m]$). Queda la pregunta: ¿es una asíntota teórica inalcanzable, o existe una construcción que lo logra? (Se retoma en la unidad 2.)

► En el parcial

Esquema típico de problema con un cifrado histórico (Hill, transposición por columnas, Jefferson, homofónico, Vigenère):

1. Calcular $\#M$, $\#K$, $\#C$ (p. ej. permutaciones $\Rightarrow n!$; rotación $\Rightarrow \#K = n - 1$; Vigenère de largo ℓ sobre n símbolos $\Rightarrow \#K = n^\ell$).
2. Verificar la condición necesaria $\#K \geq \#M$.
3. Demostrar o refutar secreto perfecto vía $\Pr[M = m \mid C = c] = \Pr[M = m]$ (o exhibiendo m, c que la violan).

Lectura recomendada: Katz & Lindell, *Introduction to Modern Cryptography*, Capítulo 1.

2. Cifrado

Esta unidad recorre el camino que va desde la **seguridad incondicional** (secreto perfecto, one-time pad) hacia la **seguridad computacional** (indistinguibilidad, CPA, CCA), y de ahí a las construcciones prácticas: cifrado de flujo, cifrado de bloque y modos de encadenamiento.

► En el parcial

Es una de las unidades **más evaluadas** del parcial. Los tres núcleos recurrentes son: (1) **demonstración formal de secreto perfecto** sobre un esquema dado; (2) **variantes de CBC y propagación de errores**; (3) **experimentos PrivK** (eav / CPA) mostrando que un esquema determinista no es seguro. Conviene dominar los tres.

2.1. Cifrado simétrico (criptosistema)

Un criptosistema es una **terna de algoritmos** (Gen, Enc, Dec):

Definición: Criptosistema simétrico

- $\text{Gen} : () \rightarrow K$ — generador de clave. En general es una función *no determinística* que elige una clave al azar del espacio de claves con distribución uniforme.
- $\text{Enc} : K \times P \rightarrow C$ — cifrado, $e_k(m) = c$.
- $\text{Dec} : K \times C \rightarrow P$ — descifrado, $d_k(c) = m$.

Propiedad básica (corrección): para todo m y k válidos

$$d_k(e_k(m)) = m.$$

Conjuntos involucrados:

- K : espacio de claves — todas las claves posibles.
- P (o M): espacio plano — todos los mensajes posibles.
- C : espacio cifrado — todos los mensajes cifrados posibles.

△ Cuidado — error típico

La propiedad básica habla solo de la *relación* entre las funciones (que Dec invierte a Enc); **no dice nada sobre seguridad**. No hay una única definición de seguridad: cada modelo (secreto perfecto, eav, CPA, CCA) define un escenario distinto.

2.2. Secreto perfecto (Shannon)

Definición: Secreto perfecto (Shannon)

Dado un criptosistema (Gen, e, d) , posee **secreto perfecto** si para toda distribución de probabilidades sobre M , para cada mensaje m y cada cifrado c con $\Pr[C = c] > 0$:

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

Intuición: **conocer el texto cifrado no aporta ninguna información extra sobre el texto plano** que no se tuviera a priori. La tabla de probabilidades de los mensajes no cambia al conocer c .

! Importante — remarcado en clase

La paradoja del secreto perfecto. Como $c = e_k(m)$, las variables aleatorias C y M están *relacionadas* por una función (no son independientes). Sin embargo, la condición de secreto perfecto, leída probabilísticamente, dice exactamente que M y C son estadísticamente *independientes*. El “truco” que lo hace posible: las ecuaciones no miran las claves; **la clave aleatoria es la que introduce esa independencia** a pesar de la relación funcional.

Definición alternativa (por encriptación). Equivalente a la de Shannon: para todo par de mensajes m, m' y todo cifrado c ,

$$\Pr[\text{Enc}_k(m) = c] = \Pr[\text{Enc}_k(m') = c].$$

Es decir, la distribución del cifrado es la misma sin importar qué mensaje se cifró.

2.2.1. Teorema (condición necesaria): $\#K \geq \#M$

! Importante — remarcado en clase

Secreto perfecto $\Rightarrow \#K \geq \#C \geq \#M$ (equivalentemente $|K| \geq |M|$).

Resultados teóricos asociados:

- Cualquier criptosistema con secreto perfecto es **reducible al OTP** (existe un isomorfismo con el one-time pad).
- Cualquier sistema que *no* sea reducible al OTP **no** posee secreto perfecto.
- Consecuencia: el secreto perfecto es demasiado impráctico; se necesitan otras construcciones.

Cuando además $\#K = \#M$, el secreto perfecto exige (teorema de Shannon):

1. las claves se eligen con distribución **uniforme**, y
2. para cada par (m, c) existe una **única** clave k tal que $e_k(m) = c$.

2.2.2. Método para demostrar secreto perfecto (PASO A PASO)

► En el parcial

Cómo se pide (2015, 2016, 2017, 2025-2): dado un esquema concreto, demostrar si tiene o no secreto perfecto. Método esperado:

1. **Calcular los cardinales $\#M, \#K, \#C$.** Si $\#K < \#M$, *ya* se concluye que NO hay secreto perfecto (por el teorema).
2. **Enunciar la definición usada:**
 - Shannon: $\Pr[M = m \mid C = c] = \Pr[M = m] \forall m, c$; o bien
 - por encriptación: $\Pr[\text{Enc}_k(m) = c] = \Pr[\text{Enc}_k(m') = c]$.
3. **Verificar el teorema:** $\#K \geq \#M$; con $\#K = \#M$ chequear claves uniformes y unicidad de la clave que lleva cada m a cada c .
4. **Concluir.** Si todo se cumple, hay secreto perfecto; si falla algún paso, exhibir el contraejemplo (un m, c donde $\Pr[M = m \mid C = c] \neq \Pr[M = m]$).

Lema auxiliar (probabilidad del cifrado). Para calcular $\Pr[C = c]$ se barren todas las claves (eventos disjuntos, porque la clave no toma dos valores a la vez):

$$\Pr[C = c] = \sum_k \Pr[C = c \cap K = k].$$

Como clave y mensaje se eligen de forma *independiente*, $\Pr[C = c \cap K = k] = \Pr[M = d_k(c)] \cdot \Pr[K = k]$.

2.2.3. Ejemplo trabajado: clave NO aleatoria (rompe el secreto perfecto)

Esquema tipo OTP sobre mensajes de 2 bits, pero con clave **sesgada**:

$$\Pr[k=00]=0,3, \quad \Pr[k=01]=0,1, \quad \Pr[k=10]=0,4, \quad \Pr[k=11]=0,2.$$

Distribución del texto plano:

$$\Pr[m=00]=0,60, \quad \Pr[m=01]=0,15, \quad \Pr[m=10]=0,10, \quad \Pr[m=11]=0,15.$$

Se intercepta $c = 01$. Queremos $\Pr[M=00 \mid C=01]$.

Usamos la igualdad de probabilidades condicionales:

$$\Pr[M=00 \mid C=01] \cdot \Pr[C=01] = \Pr[C=01 \mid M=00] \cdot \Pr[M=00].$$

(a) Calcular $\Pr[C=01]$ (sumando sobre las claves, usando $c = m \oplus k$):

$$\begin{aligned} \Pr[C=01] &= \sum_k \Pr[M=01 \oplus k] \cdot \Pr[K=k] \\ &= \Pr[M=01] \Pr[K=00] + \Pr[M=00] \Pr[K=01] \\ &\quad + \Pr[M=11] \Pr[K=10] + \Pr[M=10] \Pr[K=11] \\ &= 0,15 \cdot 0,3 + 0,6 \cdot 0,1 + 0,15 \cdot 0,4 + 0,1 \cdot 0,2 = \mathbf{0,185}. \end{aligned}$$

(b) Calcular $\Pr[C=01 \mid M=00]$. Si $m = 00$, la única clave que da $c = 01$ es $k = 01$, luego $\Pr[C=01 \mid M=00] = \Pr[K=01] = \mathbf{0,1}$.

(c) Despejar:

$$\Pr[M=00 \mid C=01] = \frac{0,1 \cdot 0,6}{0,185} = \mathbf{0,32} \neq 0,6 = \Pr[M=00].$$

△ Cuidado — error típico

Como $0,32 \neq 0,6$, **NO hay secreto perfecto**: se extrajo información (saber que $c = 01$ vuelve *menos* probable que el mensaje fuera 00). El secreto perfecto se rompe **en cuanto la clave no es uniforme**.

! Importante — remarcado en clase

Si en cambio la clave fuese uniforme ($\Pr[K=k] = 1/N$ con $N = 2^n$), se obtiene $\Pr[C=c] = 1/N$ *independientemente del mensaje*, y entonces $\Pr[M=m \mid C=c] = \Pr[M=m]$ — es el caso del OTP con secreto perfecto.

2.3. One-Time Pad (OTP) / Vernam

Atribuido a Vernam (1917). Es la transformación más simple posible y, sin embargo, alcanza el máximo teórico de seguridad.

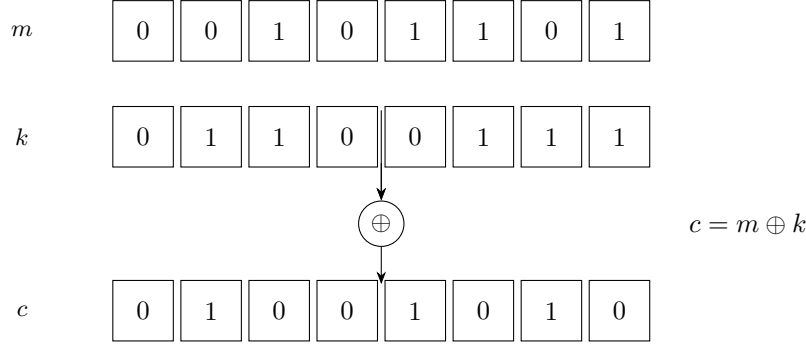
Definición: One-Time Pad

Gen : $k \leftarrow \{0, 1\}^n$, $n = |m|$ (clave uniforme)

Enc : $e_k(m) = m \oplus k$

Dec : $d_k(c) = c \oplus k$

El cifrado es un XOR bit a bit entre mensaje y clave. Descifrar es el mismo XOR (en $\mathbb{Z}_2$ el XOR es lineal e involutivo: es su propia inversa).



One-Time Pad: el cifrado es el XOR bit a bit del mensaje m con la clave k (de igual longitud), produciendo $c = m \oplus k$. Descifrar es el mismo XOR: $c \oplus k = m$.

“El OTP aplica la transformación más simple y tonta que se les puede ocurrir: un XOR bit a bit entre cada bit del mensaje y cada bit de la clave. Y ahí se acabó la magia.”

— dicho en clase

2.3.1. El OTP tiene secreto perfecto (demostración)

Sea $|k| = n$, $N = 2^n = \#K$. Queremos probar $\Pr[M=m \mid C=c] = \Pr[M=m]$.

Lema 1 — $\Pr[C=c] = 1/N$.

$$\Pr[C=c] = \sum_k \Pr[C=c \cap K=k]$$

$$\begin{aligned} \Pr[C=c \cap K=k] &= \Pr[M=c \oplus k \cap K=k] = \Pr[M=c \oplus k] \cdot \Pr[K=k] \quad (\text{indep.}) \\ &= \Pr[M=c \oplus k] \cdot \frac{1}{N}. \end{aligned}$$

Entonces

$$\Pr[C=c] = \frac{1}{N} \sum_k \Pr[M=c \oplus k] = \frac{1}{N} \cdot 1 = \frac{1}{N}.$$

La suma da 1 porque al recorrer todas las claves k , $c \oplus k$ recorre *todos* los mensajes posibles (sin repetir), y la suma de las probabilidades de todos los valores de una v.a. es 1. Notar que $\Pr[C=c]$ **no depende del mensaje**.

Parte 2 — conclusión. Por probabilidades condicionales:

$$\begin{aligned} \Pr[M=m \mid C=c] \cdot \Pr[C=c] &= \Pr[C=c \cap M=m] \\ &= \Pr[K=c \oplus m \cap M=m] \\ &= \Pr[K=c \oplus m] \cdot \Pr[M=m] \quad (\text{indep.}) \\ &= \frac{1}{N} \cdot \Pr[M=m]. \end{aligned}$$

Como $\Pr[C=c] = 1/N$ (Lema 1), se cancelan los $1/N$:

$$\boxed{\Pr[M=m \mid C=c] = \Pr[M=m]}. \quad \blacksquare$$

2.3.2. Limitaciones del OTP (las malas noticias)

! Importante — remarcado en clase

1. **Clave tan larga como el mensaje:** $\#K \geq \#C$. Para una novela de bits hace falta una clave de la misma cantidad de bits. No achica el problema: proteger la clave cuesta lo mismo que proteger el mensaje.
2. **No reutilizar la clave (one-time).** Si se cifran dos mensajes con la misma clave:

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k \Rightarrow c_1 \oplus c_2 = m_1 \oplus m_2.$$

Se cancela la clave y se filtra el XOR de los planos $\Rightarrow$ se pierde el secreto perfecto (queda como un cifrado por sustitución).

3. **La clave DEBE ser verdaderamente aleatoria.** La demostración usa $\Pr[K=k] = 1/N$ (distribución uniforme). Los generadores reales son *pseudoaleatorios* (algorítmicos, deterministas): si se rompe la hipótesis de aleatoriedad, se cae la demostración.

“En escenarios donde podamos satisfacer todas las condiciones del one-time pad, tenemos garantías absolutas de seguridad.”

— dicho en clase

Usos documentados reales: cifrado en papel en la 2.^a Guerra Mundial y el “teléfono rojo” Washington–Moscú durante la Guerra Fría (clave generada midiendo decaimiento radiactivo, distribuida en maletas). Fuera de esos escenarios acotados, es impráctico (p. ej. comercio electrónico).

2.4. Seguridad computacional

El secreto perfecto = **seguridad incondicional** (sin condiciones sobre el adversario). Como es impráctico, se relajan dos cosas para pasar a la **seguridad computacional**:

1. **Limitar escenarios:** el adversario tiene poder de cómputo muy grande pero *acotado* (especialmente en **tiempo**). Se lo modela como algoritmo **PPT** (Probabilistic Polynomial Time). Con poder de cómputo infinito, “game over”.
2. **Limitar garantías:** se acepta una **pequeña probabilidad de éxito** para el atacante (se abandona la infalibilidad). Como el cifrado deja de ser “aleatorio puro”, la fuerza bruta siempre es posible (analogía: comprar todos los números de la lotería, o tener suerte y acertar al primer intento).

Definición: Función despreciable y nivel de seguridad

El **nivel de seguridad** n relaciona la cota del poder del adversario con su probabilidad de éxito. Dado n se espera que:

- el adversario corra algoritmos de orden $\text{PPT}(n)$;
- su probabilidad de éxito $\varepsilon(n)$ sea **despreciable**: $\varepsilon(n)$ es despreciable $\iff \varepsilon(n) < 1/n^k$ (para todo polinomio, a partir de cierto n).

2.4.1. Pruebas de seguridad (experimentos)

Una prueba de seguridad es una serie de pasos que pone a prueba un **algoritmo** A que **representa un ataque**. El atacante gana o pierde; se puede repetir muchas veces e interesa su **probabilidad de éxito**.

PrivK-eav (Eavesdropping / indistinguibilidad).

Definición: Experimento $\text{Eav}_{A,\Pi}$

Dado un adversario $A(n)$ y un criptosistema $\Pi(n)$:

1. A genera m_0 y m_1 arbitrariamente.
2. Se genera una clave $k \leftarrow K$.
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = e_k(m_b)$.
5. A emite $b' \in \{0, 1\}$.

$\text{Eav}_{A,\Pi} = 1$ si $b = b'$ (A gana).

Π es **indistinguible ante observadores** si $\Pr[\text{Eav}_{A,\Pi}=1] = 0,5 + \varepsilon(n)$ con ε despreciable.

PrivK-mult (múltiples cifrados).

Definición: Experimento $\text{Mul}_{A,\Pi}$

Igual que eav pero A genera dos *vectores* de mensajes $(m_{00}, \dots, m_{0i})$ y $(m_{10}, \dots, m_{1i})$; recibe $(c_0, \dots, c_i)$ con $c_j = \text{Enc}_k(m_{bj})$; emite b' . Π es indistinguible bajo múltiples cifrados si $\Pr[\text{Mul}_{A,\Pi}=1] = 0,5 + \varepsilon(n)$.

PrivK-CPA (Chosen Plaintext Attack).

Definición: Experimento $\text{CPA}_{A,\Pi}$

1. Se genera una clave $k \leftarrow K$.
2. A obtiene un **oráculo** $f(x) = e_k(x)$ (puede cifrar lo que quiera) y genera (m_0, m_1) .
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = e_k(m_b)$.
5. A emite $b' \in \{0, 1\}$.

$\text{CPA}_{A,\Pi} = 1$ si $b = b'$. Π es **CPA-Secure** si $\Pr[\text{CPA}_{A,\Pi}=1] = 0,5 + \varepsilon(n)$.

2.4.2. Propiedades CPA y necesidad de cifrado probabilístico

! Importante — remarcado en clase

- Un criptosistema determinista **NO** puede ser CPA-Secure (ni seguro bajo múltiples cifrados). El adversario aprovecha que $e_k(x)$ es constante.
- Si un criptosistema es CPA-Secure para un mensaje, lo es para múltiples.
- Un CPA-Secure de tamaño limitado (n bits) puede extenderse: $m = m_0 \| m_1 \| \dots \| m_i$ y $\text{enc}_k(m) = \text{enc}_k(m_0) \| \dots \| \text{enc}_k(m_i)$.

► En el parcial

Por qué un esquema determinista no es CPA-seguro (ataque típico, 2018, 2023, 2025-1). El adversario:

1. Pide al oráculo $c_0 = e_k(m_0)$ y $c_1 = e_k(m_1)$ con $m_0 \neq m_1$.
2. Recibe el desafío $c = e_k(m_b)$.
3. Compara: si $c = c_0$ responde $b'=0$; si $c = c_1$ responde $b'=1$. Como el cifrado es determinista, **acierta siempre** ($\Pr = 1$).

Ejemplo: cifrado afín $c = \langle r, ar + b + m \rangle$. Si se fija $r = (a + b)$ (o cualquier r constante) el esquema se vuelve *determinista* $\Rightarrow$ el adversario pide cifrado de m_0 y m_1 , compara y gana.

Conclusión: CPA-Secure $\Rightarrow$ el cifrado debe ser **probabilístico**. Forma canónica (2016): $c = \langle r, F_k(r) \oplus m \rangle$ con r aleatorio por mensaje.

$\triangle$ Cuidado — error típico

Errores típicos:

- Olvidar que un esquema **determinista nunca es CPA-seguro**.
- **Confundir secreto perfecto con seguridad computacional:** el primero es incondicional (no asume límite de cómputo); el segundo admite adversarios PPT con probabilidad de éxito despreciable.

Evitar reutilizar la clave (nonce / IV). Para volver probabilístico al cifrado de flujo se agrega a la función generadora un valor que **no debe repetirse con la misma clave** (*nonce* o *IV*). Dos formas:

- **Modo sincronizado:** un único IV, G produce un keystream largo $G(S)$ que se va XOR-eando con $M_0, M_1, \dots$
- **Modo no sincronizado:** un IV por mensaje (IV_0 con M_0 , IV_1 con M_1 , $\dots$), generando $G(S), G(S'), \dots$

2.5. Cifrado de flujo

Intercambia la clave del OTP por la salida de un **generador pseudoaleatorio** G :

Gen : $k \leftarrow K$

Enc : $e_k(m) = G(k) \oplus m$

Dec : $d_k(c) = G(k) \oplus c$

! Importante — remarcado en clase

La gran diferencia con el OTP: $|K| \lll |M|$. Por ejemplo $|K| = 2^{128}$ y $|M| = |K|^{128}$. La clave (chica, p. ej. 128 bits) actúa como **semilla** (*seed*); G la expande a tantos bits como tenga el mensaje.

Definición: Generador pseudoaleatorio (PRG)

Es un algoritmo **determinístico** que expande una semilla y cuya salida “parece aleatoria”. Formalmente, sea $D = \{f : \{0, 1\}^n \rightarrow \{0, 1\}^s\}$ una familia de funciones. $G : \{0, 1\}^n \rightarrow \{0, 1\}^s$ con $s < n$ es PRG respecto de D si para toda $f \in D$:

$$\Pr(f(G(r^s)) \neq f(r^n)) = \varepsilon \quad (\text{despreciable}),$$

donde r^n es una secuencia *realmente* aleatoria. Misma semilla $\Rightarrow$ misma secuencia.

! Importante — remarcado en clase

Teorema: si $G(\cdot)$ es un generador pseudoaleatorio, entonces el cifrado de flujo es **indistinguible ante observadores** (pasa la prueba eav).

► En el parcial

Ejercicio recurrente: demostrar que si es posible *distinguir* $G(\cdot)$ de una secuencia aleatoria, entonces el cifrado de flujo basado en G **no pasa la prueba eav**. (Basta exhibir un *distinguisher*: mostrar que existe una función que separa la salida de G de una secuencia realmente aleatoria implica que hay ataque potencial — explotarlo es otra cuestión.)

► En el parcial

Los cifrados de flujo **NO** son seguros bajo múltiples cifrados con la misma clave (igual que el OTP reusado):

$$c_1 = m_1 \oplus G(s), \quad c_2 = m_2 \oplus G(s) \Rightarrow c_1 \oplus c_2 = m_1 \oplus m_2.$$

Ataque que gana la prueba MUL:

$$A \rightarrow (m_{00}=0 \dots 0, m_{01}=0 \dots 0), \quad (m_{10}=0 \dots 0, m_{11}=1 \dots 1).$$

A obtiene c_0, c_1 y calcula $X = c_0 \oplus c_1$. Si $X = 0 \dots 0 \Rightarrow b'=0$; si no $\Rightarrow b'=1$. Acierta siempre.

Generadores recomendados. Quebrados / no recomendados: RC4 (<https>, WEP), CSS (DVDs), A5/1 y A5/2 (GSM), E0 (Bluetooth). Recomendados: **Salsa20** ($\{0, 1\}^{128} \circ^{256} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^n$, $n = 2^{64} \cdot 2^9$) y **Rabbit** ($\{0, 1\}^{128} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^n$, $n = 2^{128}$); el segundo argumento es el IV.

2.6. Cifrado de bloque

Definición: Primitiva de cifrado de bloque

Definida para mensajes de **tamaño FIJO**: $K = \{0, 1\}^n$, $C = P = \{0, 1\}^b$. Gen : $k \leftarrow K$,
Enc : $e_k(m) = c$, Dec : $d_k(c) = m$.

Son **funciones pseudoaleatorias**: para cada k , $f(x) = \text{Enc}_k(x)$ es indistinguible de una función tomada al azar del conjunto de funciones del mismo dominio (es un PRG). Las pruebas de seguridad de los modos son **por reducción** a esta propiedad.

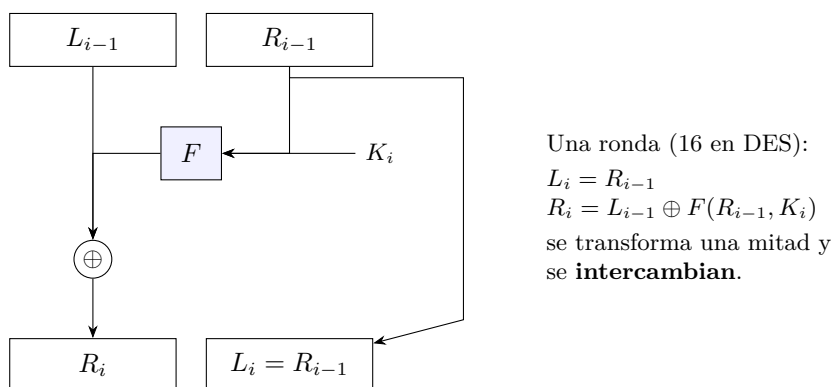
△ Cuidado — error típico

Las primitivas de bloque son **determinísticas** $\Rightarrow$ por sí solas **no pasan la prueba Mul**. Se combinan con **mecanismos de encadenamiento** (modos) para formar criptosistemas seguros. (No está demostrado que existan las funciones pseudoaleatorias.)

2.6.1. DES, 3-DES, AES

DES (Data Encryption Standard). Desarrollado por IBM, adoptado por EE.UU. (primera función de cifrado pública apoyada por un gobierno). Entrada/salida 64 bits, clave 64 bits (**56 efectivos**, 8 de paridad). Es una **red de Feistel**: se parte el bloque en dos mitades, se transforma una mitad con parte de la clave, se intercambian las mitades, y se repite la ronda 16 veces. La función de ronda: **E** (expansión 32 $\rightarrow$ 48), XOR con la subclave, **S-boxes** (sustituciones 6 $\rightarrow$ 4 bits), **P** (permutación). Subclaves: PC1 selecciona 56 bits, desplazamientos $\lll$ y PC2 genera cada subclave de 48 bits.

- Nivel teórico: 56 bits (2^{56} claves por fuerza bruta).
- 1990 — criptoanálisis diferencial: ataca DES en 2^{47} .
- 1992 — criptoanálisis lineal: ataca DES en 2^{43} .



*Estructura de Feistel de DES (una ronda): la mitad derecha pasa por la función F con la subclave K_i , el resultado se XOR-ee con la mitad izquierda, y luego se **intercambian** las mitades. Se repite 16 veces; el descifrado usa las mismas rondas con las subclaves en orden inverso.*

3-DES. Variante fuerte: $c = \text{Enc}_{k_1}(\text{Dec}_{k_2}(\text{Enc}_{k_3}(p)))$ con 3 claves independientes. Seguridad ~ 112 bits (menor a 168 por el ataque *meet-in-the-middle*). Inmune a criptoanálisis diferencial y lineal, pero $3\times$ más lento que DES.

AES. Reemplazo de DES, elegido por concurso internacional abierto. Bloques de 128 bits, clave de 128/192/256 bits. **Es la primitiva recomendada hoy.** 4 etapas por ronda: *ByteSub* (sustitución vía tabla derivada de invertir en $GF(2^8)$), *ShiftRow* (permutación), *MixColumn* (transformación lineal invertible) y *AddRoundKey* (XOR con la subclave). Asimetrías: la 1.^a ronda solo tiene *AddRoundKey*; la última no tiene *MixColumn*.

Tamaños y recomendados. Espacios de clave > 64 bits (AES: 128/192/256); espacio de mensajes ≥ 128 bits. Para comparar: partículas en el universo $\sim 2^{88}$, edad del universo $\sim 2^{58}$ s. Recomendados: **AES-CBC**, **AES-CTR** (proyectos nuevos), IDEA, 3-DES; DES queda obsoleto.

2.7. Modos de encadenamiento

Cuando el mensaje no entra en un bloque:

- **Más chico que el bloque** — *padding*. *Simple Pad*: completar con ceros (hay que conocer el tamaño real). *Des Pad*: agregar un bit 1 y luego ceros (puede agregar un bloque completo).
- **Más grande** — dividir en bloques, extender el último y transformar cada uno según un **modo de encadenamiento**.

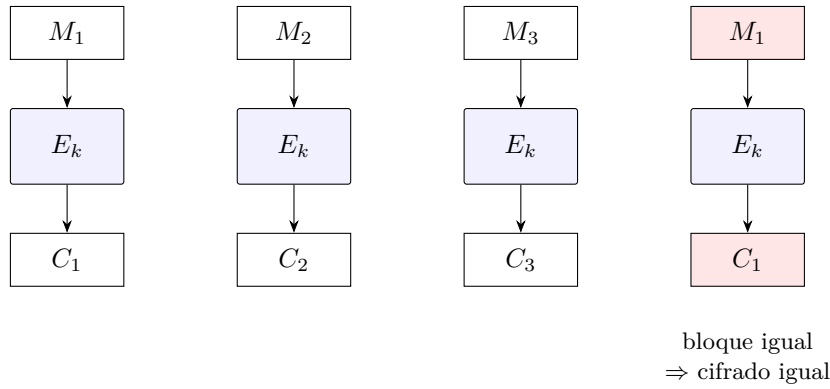
Objetivo del modo: extender una primitiva a mensajes mayores a su tamaño. Hay varios modos con propiedades distintas; no todos sirven para cada problema.

2.7.1. ECB (Electronic Codebook)

Trata el mensaje como bloques **independientes**: $c_i = e_k(m_i)$.

△ Cuidado — error típico

ECB NO es CPA-Secure (NO UTILIZAR). Al ser determinista, bloques de texto plano *iguales* producen cifrados *iguales* $\Rightarrow$ **revela patrones** (el clásico ejemplo del pingüino). Esto se pregunta en recuperatorios (2025): identificar por qué ECB filtra estructura.



*ECB: cada bloque M_i se cifra de forma **independiente** y en paralelo con la misma clave, $C_i = E_k(M_i)$. Sin encadenamiento: dos bloques de texto plano iguales producen exactamente el mismo cifrado (revela patrones).*

2.7.2. CBC (Cipher Block Chaining)

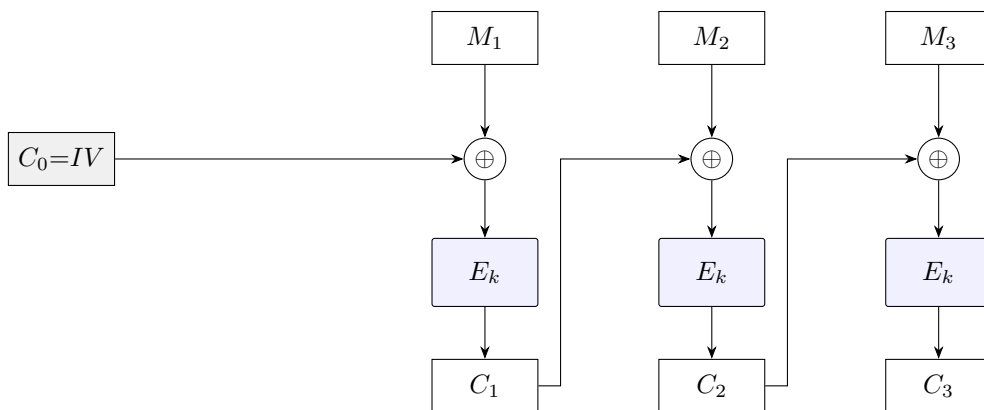
Usa la salida de un bloque como entrada del siguiente. **Disminuye el traspaso de información** y requiere un **IV aleatorio** (a menudo se lo nota $C_0 = IV$).

Definición: CBC — cifrado y descifrado

Cifrado: $C_i = E_k(M_i \oplus C_{i-1})$, $C_0 = IV$.

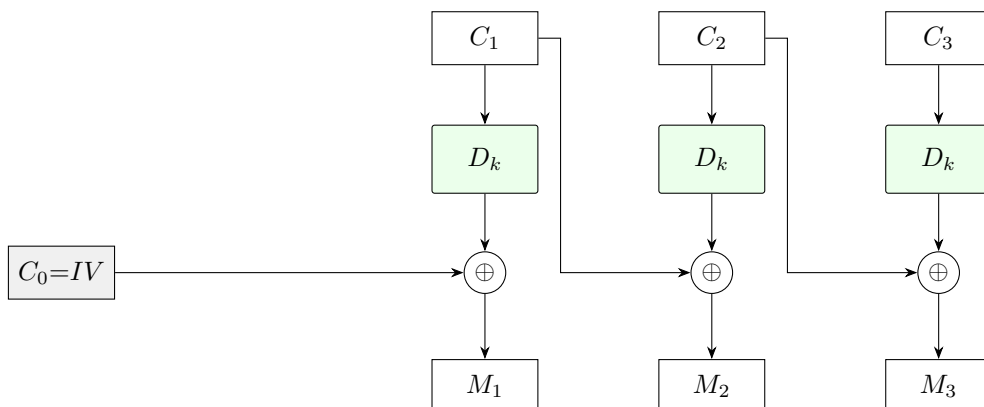
Descifrado: despejando M_i (invertir E_k con D_k y deshacer el XOR):

$$M_i = D_k(C_i) \oplus C_{i-1}.$$



CBC (cifrado): $C_i = E_k(M_i \oplus C_{i-1})$ con $C_0 = IV$. Cada bloque de texto plano se XOR-ea con el cifrado anterior antes de pasar por E_k , encadenando todos los bloques.

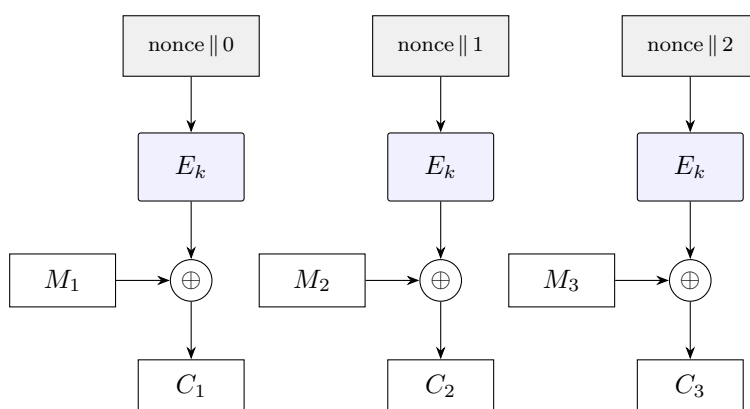
CBC con primitiva pseudoaleatoria e IV aleatorio es **CPA-Secure**.



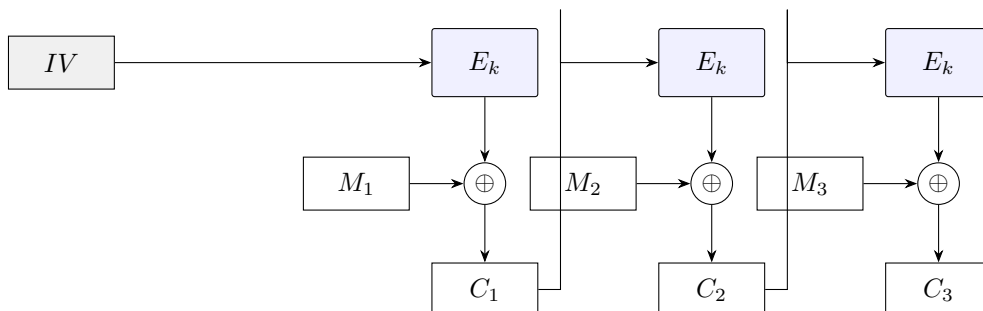
CBC (descifrado): $M_i = D_k(C_i) \oplus C_{i-1}$. El cifrado pasa por D_k y luego se le quita el encadenamiento XOR-eando con el cifrado anterior C_{i-1} .

2.7.3. Otros modos: CFB, OFB, CTR

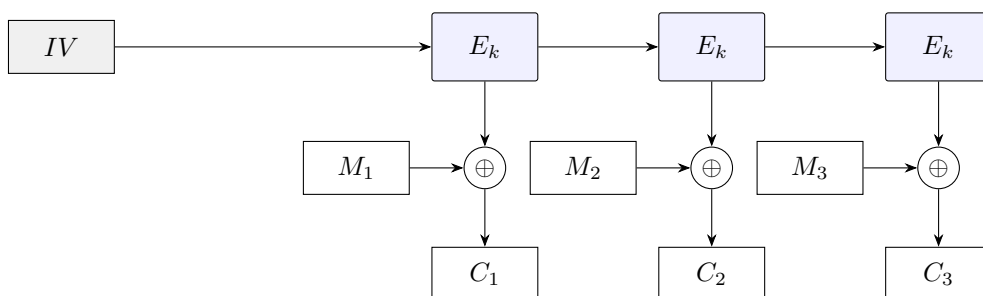
- **CFB (Cipher Feedback):** usa *solo* la función Enc (menor complejidad en dispositivos embebidos). Genera una transformación de flujo a partir de una de bloque. CPA-Secure con IV aleatorio.
- **OFB (Output Feedback):** también construye flujo a partir de bloque; permite **calcular los bits del keystream por adelantado** (no dependen del texto). CPA-Secure con IV aleatorio.
- **CTR (Counter):** usa un **contador** (nonce || contador) para generar el keystream: $c_i = E_k(\text{nonce}||i) \oplus m_i$. **Permite acceso aleatorio** y es **paralelizable** (muy útil con procesamiento paralelo). Es CPA-Secure **si no se repite el par** (k, nonce).



*CTR: el keystream se genera cifrando (nonce||i) con E_k y se XOR-ea con M_i : $C_i = E_k(\text{nonce}||i) \oplus M_i$. Cada bloque es **independiente** de los demás $\Rightarrow$ totalmente paralelizable y con acceso aleatorio. Nunca usa D_k .*



CFB (Cipher Feedback): usa solo E_k . Se cifra el bloque anterior (IV el primero, luego C_{i-1}) y la salida se XOR-ea con M_i para dar C_i . Construye un cifrado de flujo a partir de la primitiva de bloque.



*OFB (Output Feedback): la salida de E_k (el keystream) se realimenta a la entrada del bloque siguiente (no el cifrado, como en CFB). Como no depende del texto, el keystream puede **precalcularse**.*

► En el parcial

CTR descifra con la MISMA función E (2023): como el descifrado es $m_i = E_k(\text{nonce}||i) \oplus c_i$, CTR (al igual que OFB y CFB) es válido **aunque la primitiva no sea invertible** — nunca se usa D_k . Esto es un punto fino que suele preguntarse.

Resumen de seguridad de los modos.

Modo	CPA-Secure?	Notas
ECB	No	determinista, revela patrones
CBC	Sí (IV aleat.)	encadena, usa E y D
CFB	Sí (IV aleat.)	solo E ; flujo a partir de bloque
OFB	Sí (IV aleat.)	solo E ; keystream precalculable
CTR	Sí (no repetir k , nonce)	solo E ; paralelizable

2.7.4. Propagación de errores en CBC (NÚCLEO de examen)

► En el parcial

Cómo se pide (casi todos los años: 2015 Propagating CBC, 2016 FSM, 2018/2023 CBC con $C_0 = IV$, 2025-1, 2025-2). Método general:

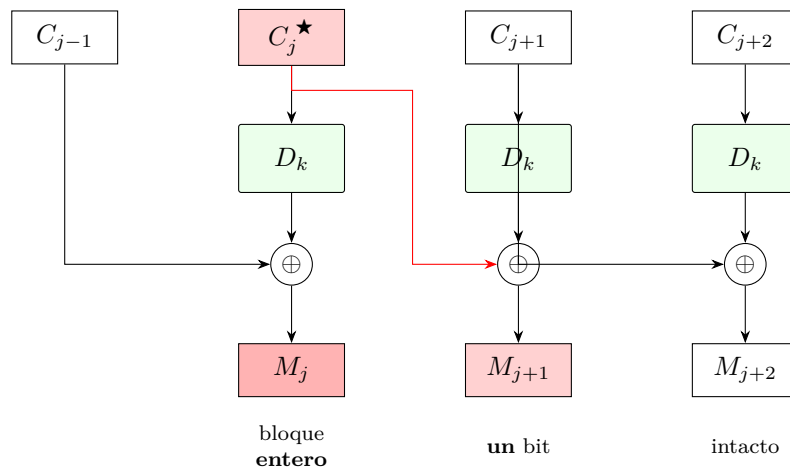
1. **Escribir la recurrencia de cifrado** del esquema dado y **despejar M_i** invirtiendo E_k (con D_k) y deshaciendo el/los XOR.
2. **Analizar la propagación de UN bit de error en C_j :** ver qué bloques de texto plano descifrado quedan corruptos.
3. **Analizar borrado/inserción de bloques:** ver qué se desincroniza.

CBC estándar — propagación de un bit de error en C_j . Como $M_i = D_k(C_i) \oplus C_{i-1}$, un error en C_j afecta a dos descifrados:

- $M_j = D_k(C_j) \oplus C_{j-1}$: el C_j corrupto **pasa por D_k** $\Rightarrow$ corrompe **todo el bloque M_j** (un bit de entrada de un cifrador de bloque cambia $\sim$ todo el bloque de salida).
- $M_{j+1} = D_k(C_{j+1}) \oplus C_j$: aquí C_j entra por **XOR directo** $\Rightarrow$ corrompe **solo el bit correspondiente** de M_{j+1} .
- M_{j+2} y siguientes: **intactos**.

! Importante — remarcado en clase

En CBC estándar el error es **autolimitado a 2 bloques**: todo M_j + un bit de M_{j+1} .



*Propagación de error en CBC: un bit erróneo en C_j (★) pasa por D_k y corrompe **todo** M_j ; además entra por XOR directo en $M_{j+1} = D_k(C_{j+1}) \oplus C_j$, corrompiendo **solo ese bit**. Desde M_{j+2} todo queda intacto: el error es **autolimitado a 2 bloques**.*

Propagating CBC (PCBC) — 2015. Variante donde el encadenamiento incluye también el texto plano, de modo que un error en C_j **se propaga a TODOS los bloques siguientes** (no se autolimita). El método es el mismo: escribir la recurrencia, despejar M_i y ver que el término corrupto nunca se cancela en los bloques posteriores.

Borrado / inserción de bloques.

- **Borrar $C_0 = IV$:** como $M_1 = D_k(C_1) \oplus C_0$, se **desincroniza** M_1 (se pierde la referencia); los demás bloques se descifran con un corrimiento.
- **Borrar el último bloque C_n :** se pierde **solo** M_n ; el resto queda intacto.

△ Cuidado — error típico

En estos ejercicios el error más común es no distinguir el camino del bloque corrupto: **pasar por D_k corrompe el bloque entero**; entrar por **XOR corrompe solo ese bit**. Tener claro *en cuántos bloques aparece C_j* en las ecuaciones de descifrado.

2.8. Estado de un criptosistema

- **Seguro:** cumple con las expectativas de su modelo de seguridad.
- **Debilitado:** existen adversarios con probabilidad no despreciable de éxito, pero el esfuerzo es enorme (décadas) o las condiciones muy difíciles (p. ej. disponer de 2^{80} mensajes).
- **Quebrado:** existen adversarios con probabilidad no despreciable de éxito en **tiempos prácticos**.

! Importante — remarcado en clase

Un criptosistema puede ser **seguro y estar quebrado al mismo tiempo**: hay múltiples pruebas de seguridad y cada una evalúa un escenario distinto.

“Se considera mala práctica desarrollar un criptosistema nuevo para un proyecto: existen funciones estudiadas por años con un nivel de confianza adecuado.”

— dicho en clase

Lectura recomendada: Katz & Lindell, Introduction to Modern Cryptography, capítulos 2 y 3.

3. MACs y Cifrado Autenticado

Hasta ahora la criptografía vista resolvía **confidencialidad** (controlar la diseminación de información). Esta unidad ataca el otro gran problema: la **integridad**, es decir, cómo confiar en que la información no fue modificada, aun cuando el origen es confiable. La primitiva básica para esto es el **MAC** (Message Authentication Code), y luego veremos funciones de hash y cómo combinar confidencialidad con integridad (cifrado autenticado).

“En general el problema de integridad no es evitar la modificación, porque muchas veces no se puede, sino detectarla.”

— dicho en clase

3.1. El nuevo escenario de ataque: integridad

El cifrado por sí solo no protege contra **modificaciones** del texto cifrado. El profesor lo motiva con un sistema de RR.HH. que guarda sueldos cifrados en una base de datos, con un criptosistema de flujo CPA-Secure.

- **No se puede extraer el valor:** aunque el atacante sepa que un sueldo es \$1.000.000, como el criptosistema es *no determinista* (usa IV), cifrar ese valor da algo distinto cada vez. No puede identificar qué fila es ni aproximar el sueldo. La parte de confidencialidad es realmente segura.
- **Pero sí se puede manipular:** un administrador que conoce su legajo (2678) y el de su jefe (2890) puede copiar el texto cifrado del jefe sobre el suyo (*update*) y obtener un aumento de sueldo, *sin conocer ningún valor*. El escenario CPA se mantiene (nunca extrajo información), pero igual rompió el sistema.

Maleabilidad. Es la propiedad de que una modificación del texto cifrado se propaga de forma sistemática/controlable al texto plano.

Definición: Maleabilidad

Cuando una modificación en el texto cifrado se puede traducir de forma sistemática en alguna modificación en el texto plano. La más simple: cambio un bit del cifrado y cambia un bit del plano.

Ataque quirúrgico sobre cifrado de flujo. Si $\text{Enc}_k(m) = G(k) \oplus m = c$, entonces definiendo $c' = c \oplus x$ resulta $\text{Dec}_k(c') = m \oplus x$. Conociendo el propio sueldo en plano ($m = 1345$) y su cifrado, el atacante hace:

$$c'' = c \oplus (1345) \oplus (100000)$$

y al descifrarse da $1345 \oplus 1345 \oplus 100000 = 100000$. Cambió el sueldo a un valor arbitrario aunque la información estuviera cifrada. Aclaración: agregar el legajo dentro del cifrado (`empleado||sueldo`) frena la copia de columnas, pero **no** frena este ataque de maleabilidad.

CCA: el modelo formal. Para modelar esto se define el escenario de **texto cifrado escogido** (Chosen Ciphertext Attack, $\text{CCA}_{A,\Pi}$). Es como CPA pero el adversario además tiene un **oráculo de descifrado** $g(x) = d_k(x)$ (no solo el de cifrado $f(x) = e_k(x)$). Restricción para no hacerlo trivial: al recibir el desafío $c = e_k(m_b)$ *no* puede pedir $g(c)$.

! Importante — remarcado en clase

Ningún criptosistema visto (ni de los no vistos) es CCA-Secure. El cifrado está pensado para confidencialidad (transferencia de información), no para manipulación. Por eso la integridad requiere otra primitiva: el **MAC**.

► En el parcial

Ejercicio de cátedra: **demostrar que el cifrado de flujo en general no es CCA-Secure**. Ayuda: considerar $m_0 = (0 \dots 0)$ y $m_1 = (1 \dots 1)$ y ver qué consultas se pueden hacer a $g(x)$ al recibir c . La idea es usar la maleabilidad ($\oplus$) para transformar c en otro cifrado consultable.

3.2. Qué es un MAC

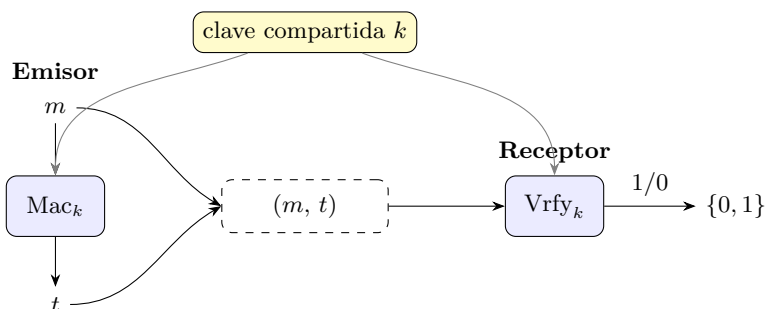
Definición: MAC (Message Authentication Code)

Terna de algoritmos (Gen, Mac, Vrfy):

- Gen : $() \rightarrow K$ — generador de clave.
- Mac : $K \times P \rightarrow T$ — etiquetador (genera la etiqueta/tag).
- Vrfy : $K \times P \times T \rightarrow \{0, 1\}$ — verificador.

Propiedad básica (correctitud): para todo m y k válidos, $\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1$.

Los conjuntos: K = espacio de claves, P = espacio de mensajes, T = espacio de etiquetas. A diferencia de los criptosistemas, hay etiquetado y verificación (no cifrado/descifrado). La verificación es parte del diseño del MAC; en muchos casos recalcula la etiqueta y compara ($\text{Mac}_k(m) == t$), pero no siempre (ver ejemplo del cifrado de la longitud).



El emisor calcula $t = \text{Mac}_k(m)$ y envía (m, t) ; el receptor corre $\text{Vrfy}_k(m, t)$ y obtiene 1 (íntegro) o 0 (alterado). La misma clave k se comparte entre ambos.

Integridad = detectar, no evitar. En un canal inseguro alguien puede pararse en el medio; no se puede impedir la modificación. Lo que da el MAC es la capacidad de **detectarla** para luego decidir qué hacer (retransmitir, alertar, abortar).

3.3. Seguridad de un MAC: infalsificabilidad (Mac-forge)

Definición: Experimento $\text{Mac-forge}_{A, \Pi}$

Dado nivel de seguridad n , adversario A y MAC $\Pi(n)$:

1. Se genera una clave $k \leftarrow K$.
2. A obtiene un oráculo etiquetador $f(x) = \text{Mac}_k(x)$ (caja negra: nunca ve k).
3. A realiza todas las evaluaciones que quiera. Sea Q el conjunto de mensajes evaluados.
4. A emite (m, t) con $m \notin Q$.

$\text{Mac-Forge}_{A, \Pi} = 1$ si $\text{Vrfy}_k(m, t) = 1$ (falsificación válida).

Si $\Pr[\text{Mac-Forge}_{A, \Pi} = 1] \leq \text{neg}(n)$ entonces Π es **infalsificable** (existencialmente infalsificable bajo ataque de mensaje elegido, CMA).

La restricción $m \notin Q$ evita el ataque trivial: si pedí el tag de “Hola” no puedo emitir “Hola” con ese tag. Una falsificación es generar, *sin conocer la clave*, un par (m, t) que pase la verificación.

Observaciones clave.

- El adversario puede obtener el MAC de cualquier mensaje que elija.
- Se considera **roto** el MAC si el adversario puede falsificar cualquier mensaje, *independientemente de si tiene sentido o no*. (Históricamente el ataque de Wang 2004 a MD5 generaba falsificaciones “sin sentido”, pero al acelerarse y poder restringir la salida a caracteres imprimibles, pasó a ser un ataque real. Lección: un ataque debilitado no se debe tomar a la ligera.)
- Demostrar que algo es infalsificable es muy difícil (hay que cubrirse de todo adversario); demostrar que es **falsificable** es más fácil: basta exhibir *un* adversario (contraejemplo).

3.4. MÉTODO para romper un MAC inseguro

► En el parcial

Pregunta recurrente (2015, 2017): romper un MAC vía Mac-forge.

Método general para mostrar que un MAC es **falsificable** (= inseguro):

1. Ubicarse como adversario A con acceso al oráculo $f(x) = \text{Mac}_k(x)$.
2. Hacer una o pocas evaluaciones inteligentes (típicamente $f(00\dots 0)$ para “leer” material de la clave/keystream, o el MAC de un prefijo).
3. **Exhibir una falsificación existencial:** construir un par (m, t) con $m \notin Q$ (distinto de lo evaluado) tal que $\text{Vrfy}_k(m, t) = 1$, sin haber consultado el tag de ese m .
4. Verificar que el par propuesto pasa la verificación.

Tres MACs de juguete (ejercicio de cátedra) y por qué fallan. (1) $\text{Mac}_k(m) = G(k) \oplus m$ (usar un cifrado de flujo como MAC). La verificación: $\text{Vrfy}_k(m, t) = (t \oplus m == G(k))$. Es **falsificable**: $G(k)$ es determinístico, así que si averiguo $G(k)$ puedo etiquetar cualquier cosa.

- Evalúo $f(00\dots 0)$. Como $G(k) \oplus 0 = G(k)$, obtengo directamente $G(k)$.
- Emito $m = 11\dots 1$ (distinto de $0\dots 0$) con tag $t = G(k) \oplus (11\dots 1)$.
- Verifica: $t \oplus m = G(k) \oplus 1\dots 1 \oplus 1\dots 1 = G(k)$. Falsificación válida.

Lección: el problema es el determinismo que se puede extraer en un mensaje y reusar en otro.

(2) $\text{Mac}_k(m) = k \oplus \text{first_k_bits}(m)$ (solo usa los primeros bits del mensaje). Doblemente falsificable:

- Evalúo $f(00\dots 0)$: como los primeros bits son 0, el tag es k . ¡Recuperé la clave! Luego emito cualquier m con tag $k \oplus \text{first_k_bits}(m)$.
- Más fácil aún: como solo mira *parte* del mensaje, tomo un m , calculo su MAC, y emito m con un carácter extra al final (la parte no chequeada). La etiqueta sigue siendo válida.

! Importante — remarcado en clase

Un MAC que **no procesa la totalidad del mensaje** es trivialmente falsificable: modifico el bit/byte que quedó afuera y reuso la misma etiqueta. Un MAC robusto **siempre** es resultado de procesar todo el contenido del mensaje.

(3) $\text{Mac}_k(m) = \text{Enc}_k(|m|)$ (cifrar la *longitud* con un cripto CPA-Secure). Como Enc es no determinista, dos mensajes de la misma longitud dan tags distintos (parece robusto desde afuera). La verificación debe *descifrar* el tag y comparar con $|m|$ (no se puede recifrar). Falsificable porque **la etiqueta no depende del contenido**, solo de la longitud:

- Pido el MAC de un mensaje de longitud 2 (p.ej. 00); obtengo un tag t tal que $\text{Dec}_k(t) = 2$.

- Emito otro mensaje distinto de la misma longitud (p.ej. 11) con el *mismo* t . Pasa la verificación porque al descifrar t vuelve a dar $2 = |11|$.
- Equivalente: pedir el etiquetado de dos mensajes de igual longitud e **intercambiar las etiquetas** (ambas validan).

► En el parcial

Falsificación por bloques (parcial 2015): dado $\text{Mac}_k(m) = \langle F_k(m_1), F_k(F_k(m_2)) \rangle$ y similares, el método es **reusar/intercambiar bloques** entre mensajes para forjar el tag de otro mensaje (los bloques se calculan de forma independiente).

MAC por bit de paridad (parcial 2017): si el tag es la paridad del mensaje, **cambiar dos bits** del mensaje conserva la paridad $\Rightarrow$ el tag sigue válido $\Rightarrow$ falsificación existencial.

3.5. Construcción de MAC: CBC-MAC

A partir de una primitiva de **cifrado de bloque** (sea F una función pseudoaleatoria) se construye un MAC infalsificable. Existe CBC-MAC con cualquier primitiva: 3DES, IDEA, AES, etc.

Definición: CBC-MAC (longitud de mensaje fija)

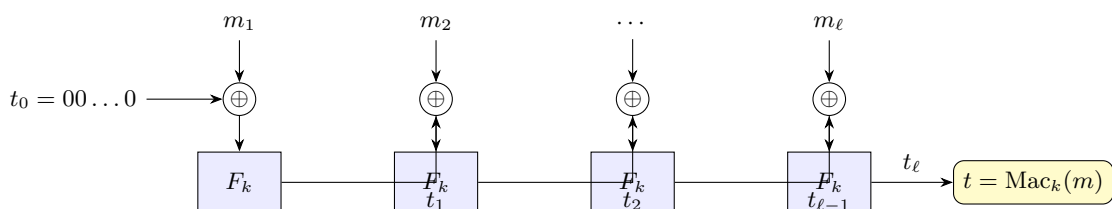
Gen : $k \leftarrow \{0, 1\}^n$.

Mac: sea $m = m_1 \| m_2 \| \dots \| m_j$, con $t_0 = 00 \dots 0$ (IV en cero, fijo):

$$t_i = F_k(t_{i-1} \oplus m_i), \quad \text{Mac}_k(m) = t_j.$$

Vrfy: $\text{Vrfy}_k(m, t) = 1 \iff t = \text{Mac}_k(m)$ (se recalcula; F_k es determinística).

Es CBC pero *sin* interés en recuperar el mensaje: se tiran todos los resultados intermedios y se conserva solo el cifrado del último bloque, que “depende” de todos los bloques anteriores por el encadenamiento. El IV se pone en cero (no estamos cifrando, no necesita ser aleatorio).



CBC-MAC: con $t_0 = 00 \dots 0$ (IV en cero), cada bloque hace $t_i = F_k(t_{i-1} \oplus m_i)$. El encadenamiento hace que el último bloque t_ℓ dependa de todo el mensaje; se descartan los intermedios y la salida es t_ℓ .

► En el parcial

¿Bajo qué condiciones es seguro CBC-MAC? (parcial 2016)

La construcción básica es infalsificable **SOLO** si se permiten mensajes de una **misma longitud fija** (ni más grandes ni más chicos), con **IV constante/cero**. Con longitud variable aparecen **ataques de extensión**.

Ataque de extensión (longitud variable). Idea de la falsificación:

- Crear dos bloques aleatorios A y B . Definir $m_1 = A$ (un bloque) y $m_2 = A \| B$.
- Calcular $t_1 = f(A)$ ($= F_k(A)$) y $t_2 = f(A \| B)$.
- Emitir el mensaje de **tres bloques** $A \| B \| (A \oplus t_1)$ con tag t_2 .

Por qué valida: el estado tras procesar los dos primeros bloques $A\|B$ es t_1 . El tercer bloque entra como $F_k(t_1 \oplus (A \oplus t_1)) = F_k(A) = t_1 = f(A) \dots$ es decir, abusando del encadenamiento se fuerza la entrada del bloque final a un valor A cuyo MAC ya se conoce. El truco general: construir mensajes por **prefijo** para acceder a los **estados intermedios** que normalmente se descartan, y luego anular el estado con $\oplus$ y forzar el valor deseado (muy parecido al ataque de maleabilidad del cifrado de flujo).

Extensiones seguras de CBC-MAC (mensajes arbitrarios). Tres formas (todas seguras; ninguna es perfecta):

1. **Clave dependiente de la longitud:** $k' = F_k(|m|)$ y usar $t_i = F_{k'}(t_{i-1} \oplus m_i)$. Así dos mensajes de distinta longitud usan claves distintas y no se aprenden los estados intermedios. (Requiere conocer $|m|$ a priori.)
2. **Longitud como prefijo:** $m' = |m| \parallel m$, y $\text{Mac}_k(m) = \text{CBC-MAC}_k(m')$. Rompe que un mensaje sea prefijo de otro. (Requiere conocer $|m|$ a priori.)
3. **Doble clave (recifrado del último estado):** k_1, k_2 ; calcular $t' = \text{CBC-MAC}_{k_1}(m)$ y luego $t = F_{k_2}(t')$. No requiere conocer $|m|$ a priori, pero usa el doble de material de clave.

△ Cuidado — error típico

Poner la longitud como **sufijo** (procesar el mensaje y agregar $|m|$ al final) **NO es seguro:** hay ataques de extensión parecidos.

3.6. Funciones de hash

Definición: Función de hash

Par de algoritmos: un selector que elige una función de una familia (información **pública**, no es clave) y la función h que convierte un mensaje arbitrario en una etiqueta (“hash” / digest / resumen) de tamaño **fijo y reducido**. **No tiene clave** (diferencia fundamental con MAC y criptosistemas).

Acepta mensajes de tamaño arbitrario (hasta terabytes) y devuelve una etiqueta chica (p.ej. 128 bits). El hash depende de **todos y cada uno** de los bits del mensaje (efecto avalancha: cambiar 1-2 bits cambia totalmente la salida). Sirve como identificador de contenido (deduplicación en backups, índice en redes P2P como eDonkey/Kademlia, etc.). No permite reconstruir el mensaje original.

Colisiones. Como hay más mensajes posibles que etiquetas, **las colisiones existen forzosa-mente** (dos mensajes distintos con el mismo hash). Como no hay clave, no se habla de “falsificación” sino de colisión.

Definición: Tres propiedades de un hash criptográfico

“Resistencia” = la única forma de hallar lo buscado es por fuerza bruta (no hay atajo algorítmico polinómico; es “computacionalmente imposible”).

- **Resistencia a preimagen:** dado un hash y , es computacionalmente imposible hallar un x con $h(x) = y$.
- **Resistencia a segunda preimagen:** dado un mensaje x , es imposible hallar un segundo $x' \neq x$ con $h(x') = h(x)$.
- **Resistencia a colisión:** es imposible hallar *dos* mensajes cualesquiera $x \neq x'$ con $h(x) = h(x')$. Es la más fuerte: implica las otras dos.

Usos.

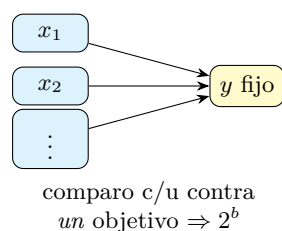
- *Resistencia a preimagen* → protocolos de **prueba de conocimiento**: demostrar que algo existe sin revelarlo (publicar el hash de una fórmula/patente y luego probar que se la tenía; juegos de cartas con compromiso).
- *Resistencia a segunda preimagen* → índices de contenido / deduplicación: dos archivos con igual contenido tienen igual hash (match semántico).

Paradoja del cumpleaños. Buscar una **colisión** es más fácil que una preimagen: al ir generando mensajes se compara cada nuevo contra *todos* los anteriores, con complejidad del orden de la **raíz cuadrada** del espacio. Medido en bits, el nivel de seguridad frente a colisiones es **la mitad**:

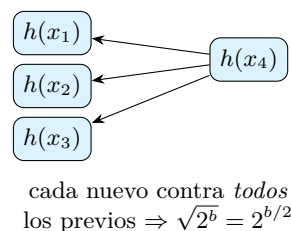
Propiedad	Objetivo	Costo fuerza bruta
Preimagen	dado y , hallar $x: h(x) = y$	$\sim 2^b$
2. ^a preimagen	dado x , hallar $x' \neq x$ con igual hash	$\sim 2^b$
Colisión	hallar cualquier par $x \neq x'$ colisionante	$\sim 2^{b/2}$

(con b = bits de salida). Por eso el nivel de seguridad de una función se mide por su resistencia a colisiones. Hoy el mínimo práctico de salida es 160 bits (colisiones $> 2^{80}$).

Preimagen / 2.^a preimagen

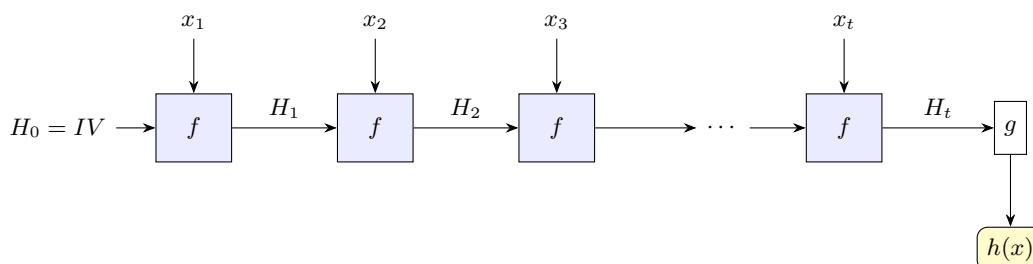


Colisión (cumpleaños)



Paradoja del cumpleaños: hallar una preimagen exige acertar contra un único objetivo ($\sim 2^b$), pero buscar una colisión compara cada hash nuevo contra todos los ya generados, por lo que basta del orden de la raíz cuadrada del espacio ($\sim 2^{b/2}$): el nivel de seguridad efectivo cae a la mitad.

Modelo general iterativo (Merkle-Damgård, 1989). La forma en que se construyen la mayoría de las funciones (MD5, SHA-1, SHA-2): un **preprocesamiento** ajusta el tamaño (padding + bloque con la longitud) y luego se aplica iterativamente una **función de compresión** f encadenando estados H_i desde $H_0 = IV$. La seguridad de toda la función la da f .



Merkle-Damgård: el mensaje formateado $x = x_1x_2 \dots x_t$ se procesa bloque a bloque por la función de compresión f , encadenando H_i desde $H_0 = IV$; una transformación final g produce la salida $h(x) = g(H_t)$.

Funciones en la práctica.

- **MD5**: *quebrada*, no usar (aparece en protocolos viejos).
- **SHA-1** (160 bits): borderline, complicada.
- **SHA-2**: durante mucho tiempo la recomendada (3 variantes), pero ~ 1 orden de magnitud más lenta que SHA-1.

- **SHA-3 (Keccak)**: ganador del concurso abierto, mismos tamaños que SHA-2 pero velocidad de SHA-1 y estructura distinta. **Estándar recomendado hoy**; 256 bits “alcanza y sobra”. SHA = Secure Hash Algorithm (estándares del NIST).

△ Cuidado — error típico

Errores típicos de Verdadero/Falso (parciales 2025):

- “Un hash sirve para encriptar” → **FALSO** (Recup. 2025). Un hash *no es reversible*: no permite recuperar el mensaje, no encripta.
- “MD5 es un criptosistema asimétrico” → **FALSO** (2025-2). MD5 es una función de *hash*, no un criptosistema asimétrico.

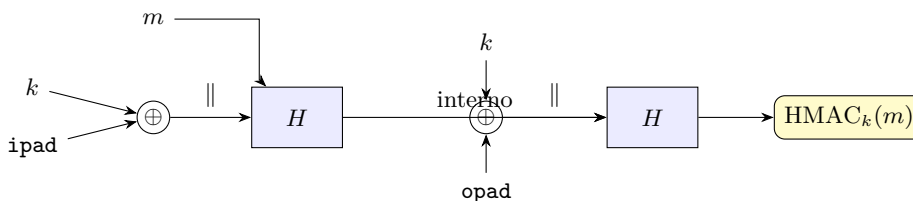
3.7. HMAC: MAC a partir de una función de hash

La segunda gran forma de construir un MAC infalsificable es a partir de una función de hash libre de colisiones. Los HMAC suelen ser 1-2 órdenes de magnitud **más rápidos** que un CBC-MAC (los hash están diseñados para procesar grandes volúmenes muy rápido), por eso se prefieren a escala.

Definición: HMAC (especificado en una RFC)

Dada una función de hash H y una clave k , con constantes públicas fijas **ipad** y **opad**:

$$\text{HMAC}_k(m) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m)).$$



HMAC: hash interno $H((k \oplus \text{ipad}) \parallel m)$ y luego hash externo $H((k \oplus \text{opad}) \parallel (\text{interno}))$. El mensaje se procesa una sola vez (el externo solo toca la etiqueta interna), por eso es muy eficiente.

La clave concatenada como prefijo rompe la posibilidad de aprender estados intermedios (misma idea que en CBC-MAC). El mensaje se procesa una sola vez (el hash exterior solo procesa la etiqueta interna), por lo que es muy eficiente. Hay **demostración formal**: si H es libre de colisiones, HMAC es infalsificable. En sistemas se ven HMAC-SHA1, HMAC-MD5 (evitar), HMAC-SHA3.

► En el parcial

Validar sin exponer el secreto (parcial 2017, caso CVV). Se quiere verificar un dato sensible (p.ej. el CVV de una tarjeta) sin almacenarlo ni transmitirlo en claro: se usa un **HMAC con clave simétrica**. Se guarda/compara el tag en vez del valor; quien no tiene la clave no puede generar ni validar el tag.

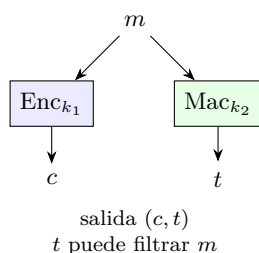
3.8. Cifrado autenticado: confidencialidad + integridad

Cuando se necesitan **las dos cosas** (confidencialidad e integridad) hay tres formas de combinar un criptosistema con un MAC. Dos son seguras, una no:

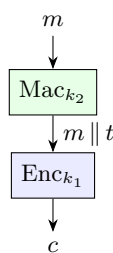
Construcción	Idea	¿Seguro?
Encrypt-and-MAC (en paralelo)	cifrar m por un lado y etiquetar m por el otro	NO
MAC-then-Encrypt (autenticar y luego cifrar)	etiquetar m , luego cifrar m y el tag juntos	<i>Puede</i>
Encrypt-then-MAC (cifrar y luego autenticar)	cifrar m , luego etiquetar el <i>texto cifrado</i> c	SÍ

- **Encrypt-and-MAC (paralelo) — inseguro:** no hay garantía de que la etiqueta del mensaje no revele información sobre m . La seguridad de un MAC cubre infalsificabilidad, *no* confidencialidad. Publicar $t = \text{Mac}(m)$ junto al cifrado puede filtrar información del plano.
- **MAC-then-Encrypt — puede ser seguro:** se cifra también el tag, así su posible filtración queda protegida. Lo usa SSH. Pero **no hay prueba genérica**; solo pruebas para combinaciones concretas, y hay que tener cuidado si el MAC es un CBC-MAC con la misma primitiva de cifrado (simetrías explotables).
- **Encrypt-then-MAC — siempre seguro:** hay **demostración general**. Es lo que se llama **cifrado autenticado**.

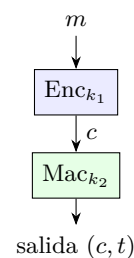
Encrypt-and-MAC
INSEGURO



MAC-then-Encrypt
puede ser seguro



Encrypt-then-MAC
SÍ (cifrado autenticado)



MAC sobre el cifrado c
claves $k_1 \neq k_2$

Las tres construcciones. **Encrypt-and-MAC** (en paralelo): inseguro, el tag de m puede filtrar información. **MAC-then-Encrypt**: se cifra $m || t$, puede ser seguro pero sin prueba genérica. **Encrypt-then-MAC**: se etiqueta el texto cifrado c con claves independientes $k_1 \neq k_2$; siempre seguro y es el “cifrado autenticado”.

Definición: Cifrado autenticado (Encrypt-then-MAC)

Requiere un criptosistema **CPA-Secure** y un MAC **infalsificable**, con **claves independientes** ($k_1 \neq k_2$). Se construye un criptosistema nuevo:

- Gen: genera ambas claves (k_1 del cifrado, k_2 del MAC).
- Cifrado: se cifra m obteniendo c , y se acompaña con la **etiqueta del texto cifrado** $t = \text{Mac}_{k_2}(c)$.
- Descifrado/verificación: **primero se verifica** el tag sobre c ; si pasa, se descifra; si no, la operación **falla** (error/excepción).

! Importante — remarcado en clase

Encrypt-then-MAC = MAC sobre el CIFRADO c , con claves independientes $k_1 \neq k_2$. Aparece en parciales 2016, 2018, 2023. La novedad: si alguien manipula el texto cifrado, el descifrado **falla** (antes solo quedaba “basura” según la maleabilidad).

► En el parcial

V/F (parciales 2018, 2023, 2016): “Cifrar y al mismo tiempo hacer MAC de m con $k_1 = k_2$ ” (Encrypt-and-MAC con clave única) es **INSEGURO** $\Rightarrow$ la afirmación de que es seguro es **FALSA**. El orden correcto es MAC sobre el cifrado c , con claves **independientes**.

► En el parcial

Construcción analizada a mano (parcial 2025-1): $c = E_{k_1}(m \parallel H(k_2 \parallel m))$. Hay que describir los pasos del receptor: descifra c con k_1 , separa m y el hash adjunto, recalcula $H(k_2 \parallel m)$ con su clave k_2 y compara. Si coincide, el mensaje es **íntegro y autenticado** (solo quien tiene k_2 pudo generar ese “HMAC casero”). Da confidencialidad (E_{k_1}), integridad y autenticación, pero **NO da no-repudio** (clave simétrica compartida).

Modos AEAD en la práctica. Modos de encadenamiento que convierten una primitiva de bloque en cifrado autenticado, sin el overhead de doble función/doble clave:

- **CCM** (Counter with CBC-MAC): sigue la variante MAC-then-Encrypt (tiene prueba específica). Calcula el CBC-MAC y lo cifra junto al mensaje en modo Counter; pasa una sola vez por el mensaje pero procesa cada bloque dos veces. Usa una sola clave. Seguro si el IV y el nonce no se repiten.
- **GCM** (Galois Counter Mode): usa modo Counter (cifra cada bloque una vez) y combina los resultados con una función de hash rápida (GHASH, aritmética de polinomios). Mucho más rápido que CCM. **AES-GCM** es la recomendación práctica: rápido, estandarizado, libre de patentes, en todas las librerías serias.

! Importante — remarcado en clase

“Si se tienen que llevar algo: en sistemas reales usen criptosistemas autenticados.”

— dicho en clase En el 99 % de los casos, si se necesita confidencialidad también se necesita integridad (el cifrado común puede ser manipulado). Recomendación concreta: **AES en modo GCM**.

3.9. No-repudio: por qué los MAC no lo dan

△ Cuidado — error típico

ERROR TÍPICO MUY IMPORTANTE (parciales 2025-1 Ej.3d, 2025-2 Ej.5b): afirmar que un MAC/HMAC da **NO-REPUDIO**.

NO lo da. La clave simétrica es **compartida**: ambas partes (emisor y receptor) tienen la misma clave y *ambas* pueden generar un tag válido. Por lo tanto, el receptor no puede probar ante un tercero que el mensaje lo originó el emisor (él mismo pudo haberlo generado). El MAC da **integridad y autenticación de origen** entre las partes que comparten la clave, pero **no** no-repudio.

El **no-repudio** requiere **firma digital** (criptografía **asimétrica**: clave privada que solo posee el firmante).

4. Cifrado Asimétrico y Firma Digital

Las clases anteriores resolvían confidencialidad (criptosistemas) e integridad (MACs/hash) desde el modelo **simétrico**: emisor y receptor comparten una *misma* clave. Esta unidad revisita los mismos problemas desde el modelo **asimétrico**, donde hay *dos* claves distintas. Esto resuelve un problema que antes era imposible (el intercambio de claves) y da origen a tres construcciones nuevas: **intercambio de claves** (Diffie-Hellman), **criptosistemas asimétricos** (RSA, ElGamal) y **firmas digitales**.

“Hoy vamos a visitar la criptografía, pero desde otro modelo mental que es el modelo asimétrico.”
— dicho en clase

4.1. El problema de distribución de claves

La seguridad de toda función criptográfica depende de que un atacante **no tenga acceso a la clave**. Consecuencia práctica: las claves **no** pueden transmitirse por un canal inseguro (la información cifrada sí, las claves no).

- **Dos puntos:** se usa un canal alternativo seguro (encuentro físico, correo caro, o seguridad por capas con un canal independiente).
- **Múltiples puntos:** para n entidades que se comunican de a pares hace falta una clave por combinación, del orden de $\binom{n}{2} = O(n^2)$ claves; cada parte administra $n - 1$ claves. No escala (ej.: 100 000 empleados $\Rightarrow$ 100 000² claves, cada uno conoce 999 999).
- **Punto único de confianza (TTP):** un tercero de confianza *Trusted Third Party*, también llamado **KDC** (Key Distribution Center). Comparte una clave con cada entidad ($k_a, k_b, k_c, \dots$); si A quiere hablar con C, el KDC genera una clave de sesión k_s y se la manda a cada uno cifrada con k_a y k_c . Permite n claves para n entidades, pero es un **único punto de falla**. Base de Kerberos y Active Directory.

4.2. La idea de la criptografía asimétrica

Diffie y Hellman (1976, “We stand today on the brink of a revolution in cryptography”) parten de una analogía: hay problemas con naturaleza **asimétrica**, fáciles en un sentido y difíciles en otro (cerrar un candado es fácil, abrirlo sin llave es difícil; el problema **SAT** es difícil de resolver pero fácil de verificar).

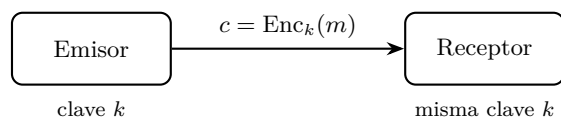
La idea: usar **dos claves independientes** (conocer una no permite derivar la otra), una para cifrar y otra para descifrar. Si un atacante tiene la clave de cifrado, igual no puede recuperar el mensaje. Entonces esa clave de cifrado se puede **publicar a propósito** $\Rightarrow$ criptografía de *clave pública*. Esto resuelve elegantemente la distribución de claves: B publica su clave de cifrado por un canal inseguro, A la usa para cifrar, y solo B (que tiene la clave de descifrado) puede recuperar el mensaje.

! Importante — remarcado en clase

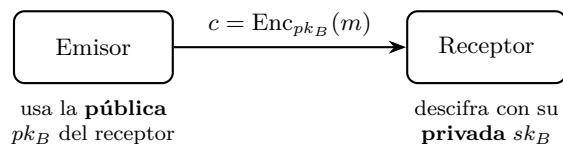
Propiedades de las claves (memorizar):

- La clave **privada/secreta** se mantiene **secreta**, fuera del alcance de atacantes.
- La clave **pública** se **distribuye** (incluso al atacante), pero de forma **autenticada** (saber que es de quien dice ser).
- El uso de las claves **no es intercambiable**. En un criptosistema se cifra solo con la pública y se descifra solo con la privada. Usarlas al revés rompe la confidencialidad.

Simétrico (misma clave)



Asimétrico (dos claves)



Simétrico: emisor y receptor comparten la misma clave k . Asimétrico: el emisor cifra con la clave pública del receptor y solo el receptor descifra con su clave privada.

Problemas difíciles que la sustentan. Toda esta criptografía se apoya en problemas matemáticos sin solución eficiente conocida:

- **Factorización** de $n = p \cdot q$ (sustenta RSA).
- **Logaritmo discreto** (DLP) en grupos finitos (sustenta DH y ElGamal).

“No eficiente” significa que las mejores soluciones son exponenciales o sub-exponenciales (equivalente a fuerza bruta).

△ Cuidado — error típico

La computación cuántica afecta a **toda** esta parte asimétrica: el **algoritmo de Shor** resolvería factorización y log discreto eficientemente (pertenecen a la misma categoría de problemas). La criptografía simétrica clásica no se ve afectada de la misma forma. Existen algoritmos post-cuánticos de intercambio de claves para reemplazarlos.

4.3. Repaso de álgebra y aritmética

- **Grupo** $(G, +)$: clausura, asociatividad, neutro, inverso. **Abeliano** si además es conmutativo. **Anillo** $(G, +, *)$: $(G, +)$ abeliano + clausura/asociatividad de $*$ + distributividad. **Cuerpo/campo**: anillo con $*$ conmutativa, neutro e inverso de $*$.
- **Grupo cíclico** $G = (\{g^n \mid n \in \mathbb{Z}\}, +)$. Grupo canónico $\mathbb{Z}_n = (\{0, 1, \dots, n-1\}, +)$. Un **generador** g es coprimo con n ; hay $\Phi(n)$ generadores. g es **primitivo** si $\text{ord}(g) = n$ (recorre todos los elementos).
- **Aritmética modular**: $a \equiv b \pmod n \Leftrightarrow a - b = k \cdot n$. Si n es primo $\Rightarrow$ campo finito $\mathbb{Z}_p$; si no, anillo (y $\mathbb{Z}_n^*$ con los coprimos a n forma grupo multiplicativo).
- **Euler / Fermat**: si $(k : n) = 1$ entonces $\exists k^{-1}$ tal que $k \cdot k^{-1} \equiv 1 \pmod n$, y

$$a^{\Phi(n)} \equiv 1 \pmod n,$$

donde $\Phi(n) = |\mathbb{Z}_n^*|$. Caso primo (Fermat): $a^{p-1} \equiv 1 \pmod p$.

- **Cálculo de Φ** : $\Phi(n \cdot m) = \Phi(n)\Phi(m)$ si $(n : m) = 1$; $\Phi(p^k) = p^{k-1}(p-1)$ con p primo. En RSA: $\Phi(p \cdot q) = (p-1)(q-1)$.

4.4. Intercambio de claves

Definición: Protocolo de intercambio de claves

Protocolo $\Pi(n)$ ejecutado por dos partes, sin entrada salvo el parámetro de seguridad. Su salida es: una **transcripción** (mensajes intercambiados, vistos por el atacante), una clave k_a que deriva una parte y una clave k_b que deriva la otra, cada una con la información de su propio extremo. **Condición fundamental:** $k_a = k_b$.

Tiene que ser la misma clave porque son dos entidades que no comparten nada de antemano y luego necesitan una clave común para usar criptografía simétrica.

Seguridad (ataques pasivos): Key-Exchange experiment $KE_{A,\Pi}$.

1. Se ejecuta Π , sea $k = k_a = k_b$.
2. Se genera $b \leftarrow \{0, 1\}$.
3. Si $b = 0 \Rightarrow k' \leftarrow \{0, 1\}^n$ (aleatoria); si $b = 1 \Rightarrow k' = k$.
4. A recibe $Trans$ y k' , y emite $b' \in \{0, 1\}$.

$KE_{A,\Pi} = 1$ si $b = b'$. Si $\Pr[KE_{A,\Pi} = 1] < \frac{1}{2} + \varepsilon$, Π es seguro. No se le pide al atacante *adivinar* la clave, sino distinguir la clave real de una aleatoria; así garantizamos que no obtiene *ninguna* información parcial. Adivinar al azar o sesgar da siempre ≈ 0.5 ; si llega a ~ 0.6 es que encontró algo y el protocolo falla.

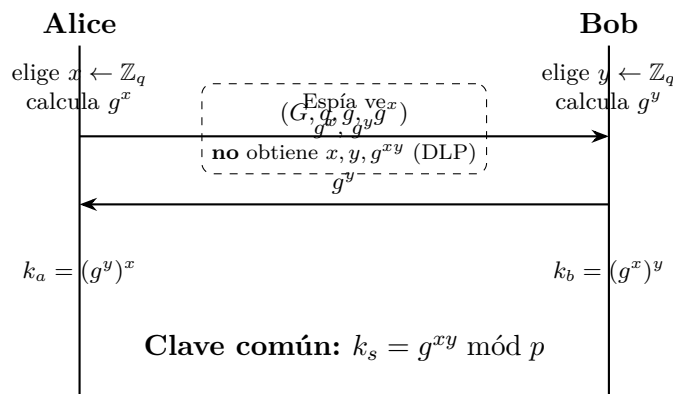
4.4.1. Protocolo Diffie-Hellman

1. A define (G, q, g) : G grupo, q su tamaño, g un generador. (Info **pública**, parte del transcript; un atacante la conoce.)
2. A elige $x \leftarrow \mathbb{Z}_q$ y calcula $h_1 = g^x$.
3. A envía a B: (G, q, g, h_1) .
4. B elige $y \leftarrow \mathbb{Z}_q$ y calcula $h_2 = g^y$.
5. B envía a A: (h_2) .
6. A calcula $k_a = h_2^x = (g^y)^x$.
7. B calcula $k_b = h_1^y = (g^x)^y$.

La clave común (de sesión) es

$$k_s = (g^x)^y \text{ mód } p = (g^y)^x \text{ mód } p = g^{xy} \text{ mód } p,$$

e $k_a = k_b$ porque la exponenciación sucesiva es conmutativa. Lo que viaja son g^x y g^y ; x **nunca** sale de A y y **nunca** sale de B.



Diffie-Hellman: Alice y Bob intercambian g^x y g^y y cada uno deriva la misma clave g^{xy} . Un espía pasivo ve g^x y g^y pero no puede recuperar x , y ni g^{xy} (problema del logaritmo discreto).

► En el parcial

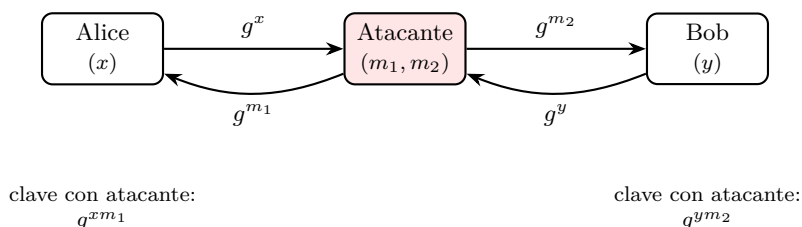
RSA pregunta por DH casi siempre así:

- **Detallar el protocolo:** $g, q, x \rightarrow g^x, y \rightarrow g^y$, clave común g^{xy} .
- **V/F “DH permite ENVIAR una clave a Bob” → FALSO.** DH **acuerda/deriva** una clave compartida *en línea*; **no transporta** una clave preexistente. (2018/2023.) No confundir acuerdo de clave con transporte de clave.
- **Seguridad = problema del logaritmo discreto (DLP).** Dado g^x y g^y no se puede obtener x ni y (condición *necesaria*). Formalmente requiere además la **conjetura de decisión DH**: dados g, g^x, g^y , un adversario no puede distinguir g^{xy} de un valor aleatorio.
- **Valores válidos de q :** se trabaja con grupos multiplicativos, y para evitar líos con factores de primalidad se elige q **primo**, así todos los elementos son coprimos. (Recup 2025, 2025-1.)

Por qué es difícil. Calcular logaritmos en campos *infinitos* es eficiente (logaritmo natural, series de Taylor). En campos *finitos* (módulo algo) el “efecto de envolvente” lo rompe: $3^2 \bmod 7 = 2$ (no 9), y recuperar x de $g^x \bmod p$ es el **problema del logaritmo discreto**, sin solución eficiente (problema abierto de más de un siglo).

△ Cuidado — error típico

DH puro no tiene autenticación. La versión original requiere un canal de transmisión **autenticado**. Solo es seguro frente a atacantes **pasivos**. Un atacante **activo** que modifica mensajes en tránsito hace un ataque **Man-in-the-Middle (MitM)** y obtiene las claves. Por eso en la práctica DH se complementa con **MACs o firmas digitales**.



MitM sobre DH puro (sin autenticación): el atacante intercepta y negocia una clave distinta con cada parte (g^{xm_1} con Alice, g^{ym_2} con Bob), descifrando y reenviando todo el tráfico.

4.5. Criptosistema asimétrico

Definición: Criptosistema asimétrico

Terna de algoritmos:

- | | |
|--|---|
| $\text{Gen} : () \rightarrow PK \times SK$ | (genera par de claves pública/privada) |
| $\text{Enc} : PK \times P \rightarrow C$ | (cifra con la pública) |
| $\text{Dec} : SK \times C \rightarrow P$ | (descifra con la privada) |

Propiedad básica: para todo m y (pk, sk) válidos, $d_{sk}(e_{pk}(m)) = m$.

Prueba de indistinguibilidad ($Eav_{A,\Pi}$). Igual que en el mundo simétrico, pero **se le da pk al adversario**: (1) $(pk, sk) \leftarrow K$; (2) A obtiene pk y genera (m_0, m_1) ; (3) $b \leftarrow \{0, 1\}$; (4) A recibe $c = e_{pk}(m_b)$; (5) A emite b' . Gana si $b = b'$; es indistinguible si $\Pr[Eav_{A,\Pi} = 1] < \frac{1}{2} + \varepsilon$.

! Importante — remarcado en clase

Como A conoce pk , puede cifrar cualquier mensaje (tiene acceso a la función de cifrado, que es justo el oráculo del test CPA). Por eso, en el mundo asimétrico:

indistinguible (pasivo) $\Rightarrow$ CPA-Secure

y son pruebas equivalentes. Y como CPA-Secure **requiere cifrado no determinístico**, **no existen criptosistemas asimétricos seguros que sean determinísticos**: hay que incluir no-determinismo desde el principio.

4.5.1. RSA (Rivest, Shamir, Adleman)

Primer criptosistema asimétrico famoso. La parte compleja es la **generación de claves**; cifrar y descifrar son simples.

“Textbook” RSA (determinístico, solo conceptual).

Elegir p, q primos, $n = p \cdot q$.

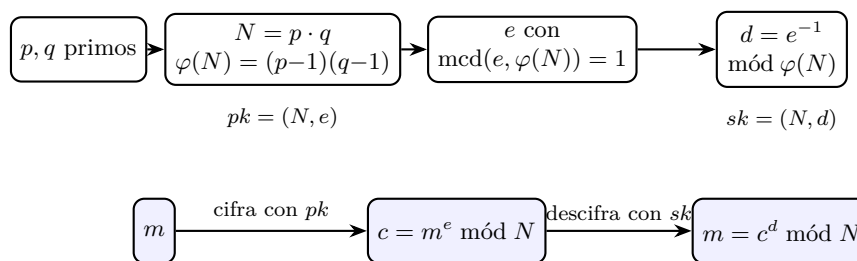
$e \leftarrow (0, \Phi(n)) \mid \text{mcd}(e, \Phi(n)) = 1$.

Calcular $d \mid e \cdot d \equiv 1 \text{ mód } \Phi(n)$ (inverso multiplicativo, Euclides extendido).

$pk = (n, e)$, $sk = (n, d)$.

$\text{Enc}_{pk}(m) \equiv m^e \text{ mód } n$, $\text{Dec}_{sk}(c) \equiv c^d \text{ mód } n$.

El cálculo $m^e \text{ mód } n$ se hace con **exponenciación rápida**, reduciendo módulo n en cada paso, para no manejar números gigantes.



RSA: arriba la generación de claves $(p, q \rightarrow N, \varphi(N), e, d)$; abajo cifrado $c = m^e \text{ mód } N$ con la pública y descifrado $m = c^d \text{ mód } N$ con la privada.

► En el parcial

RSA es tema central de parcial. Hay que saber de memoria:

- Generación: $n = pq$, $\Phi(n) = (p-1)(q-1)$, elegir e coprimo con $\Phi(n)$, $d = e^{-1} \text{ mód } \Phi(n)$.
- Cifrado: $c = m^e \text{ mód } n$. Descifrado: $m = c^d \text{ mód } n$.

Por qué funciona (Dec inversa de Enc).

$$\text{Dec}(\text{Enc}(m)) = (m^e)^d \bmod n = m^{ed} \bmod n.$$

Como $e \cdot d \equiv 1 \bmod \Phi(n)$: $e \cdot d = c \cdot \Phi(n) + 1$ (para alguna constante c).

$$m^{ed} = m^{c\Phi(n)+1} = (m^{\Phi(n)})^c \cdot m \bmod n.$$

Por Euler $m^{\Phi(n)} \equiv 1$: $= 1^c \cdot m = m \bmod n$.

Problemas del textbook RSA.

- **Es determinístico** $\Rightarrow$ no pasa la prueba de indistinguibilidad.
- **e y m chicos:** si $m^e < n$ nunca se reduce, y el log discreto coincide con el log natural (calculable). Históricamente se usaba $e = 3$.
- **Módulos repetidos:** si dos pares de claves comparten el mismo n , se puede recuperar n y luego la clave privada.
- Los primos p, q deben ser “primos fuertes”, no cualesquiera.

Ejemplo numérico (parámetros chicos).

$$p = 2357, q = 2551, n = p \cdot q = 6\,012\,707, \quad \Phi(n) = (p-1)(q-1) = 6\,007\,800.$$

$$e = 3\,674\,911 \text{ (al azar)}, \quad d = 422\,191 \text{ (Euclides ext.)}.$$

$$\text{Enc}(m = 5\,234\,673) = 5\,234\,673^{3\,674\,911} \bmod 6\,012\,707 = 3\,650\,502.$$

$$\text{Dec}(3\,650\,502) = 3\,650\,502^{422\,191} \bmod 6\,012\,707 = 5\,234\,673.$$

PKCS #1 v1.5: RSA con padding aleatorio. Estándar de RSA Labs que hace RSA **no determinístico**. Sea k la longitud de n en bytes; solo se cifran mensajes de hasta $k - 11$ bytes (sea D la longitud de m):

$$m' = 00000000 \parallel 00000010 \parallel r \parallel 00000000 \parallel m,$$

donde r son $k - D - 3$ bytes **aleatorios distintos de 0** (para sacar el padding al descifrar buscando el byte 00). Esto da ≥ 8 bytes aleatorios ($\geq 2^{64}$ variantes) $\Rightarrow$ misma clave/mensaje produce salidas distintas. Al descifrar se recupera m' y se quita el padding.

► En el parcial

RSA con padding aleatorio (PKCS #1 v1.5) se cree que es **CPA-Secure**, pero **no es CCA-Secure** (se encontraron ataques de texto cifrado elegido; es manipulable, no da integridad ni es cifrado autenticado). (2025-2.) Es decir: el padding aleatorio lo hace seguro frente a CPA, no frente a CCA.

Tamaño de claves y uso práctico. El nivel de seguridad es relativo al tamaño de los conjuntos (en RSA, $n = pq$). Como factorizar/log discreto son sub-exponenciales (más eficientes que fuerza bruta), hacen falta claves grandes: campos numéricos $n \geq 1024$ bits (hoy se recomiendan 1536 o 2048). RSA es 1-2 órdenes de magnitud más lento que AES y solo cifra mensajes acotados, por eso se suele usar para **cifrar una clave simétrica** y con ella el mensaje largo (lo mejor de los dos mundos).

4.5.2. ElGamal (basado en DH)

ElGamal es como un Diffie-Hellman “desacoplado en el tiempo” convertido en criptosistema.

Gen: Seleccionar G, q, g (campo G de tamaño q); $x \leftarrow \mathbb{Z}_q$, $h = g^x$.

$pk = (G, q, g, h)$, $sk = (G, q, g, x)$.

$Enc_{pk}(m) : y \leftarrow \mathbb{Z}_q$, $c = (c_1, c_2) = (g^y, h^y \cdot m)$.

$Dec_{sk}(c) = c_2 / c_1^x$.

$c_1 = g^y$ es “la segunda parte de DH”; $h^y = (g^x)^y = g^{xy}$ es la clave de sesión que enmascara a m (como un one-time pad). Al descifrar, $c_1^x = g^{yx}$ es esa misma clave, y dividir la cancela.

- **Resultado teórico:** si la prueba de decisión DH es difícil en G , ElGamal es **CPA-Secure**.
- **Diferencias con RSA:** ElGamal es **probabilístico** por naturaleza (elige y al azar al cifrar); permite reutilizar (G, q, g) ; **no está limitado a campos numéricos** (sirve con anillos de polinomios / campos de Galois con 2^n elementos, y con **curvas elípticas**, donde el problema de decisión DH es más complejo). En ElGamal el nivel de seguridad es relativo a $n = q$.
- **Curvas elípticas (ECC):** para seguridad equivalente a RSA-2048 (≈ 128 bits) bastan ≈ 320 bits (vs. 2048). Claves más chicas, operaciones más rápidas. Cuando un algoritmo de clave pública lleva “EC”, suele ser ElGamal sobre curva elíptica.

4.6. Firmas digitales

Son el equivalente asimétrico de los MACs (su objetivo es la **integridad**), pero con ventajas que justifican el nombre.

Definición: Firma digital

Terna de algoritmos:

$Gen : () \rightarrow PK \times SK$

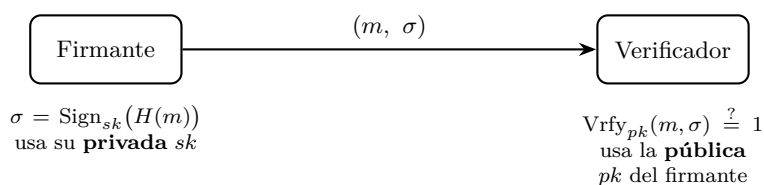
$Sign : SK \times P \rightarrow S$ (firma con la **privada**)

$Vrfy : PK \times P \times S \rightarrow \{0, 1\}$ (verifica con la **pública**)

Propiedad básica: $Vrfy_{pk}(m, Sign_{sk}(m)) = 1$.

! Importante — remarcado en clase

El rol de las claves está **al revés** que en el cifrado: se **firma con la privada** y se **verifica con la pública**. Si firmáramos con la pública, cualquiera podría firmar y se perdería la integridad.



Firma digital: el firmante firma $H(m)$ con su clave privada y cualquiera verifica con la pública. Es el rol inverso al cifrado, donde se cifra con la pública y se descifra con la privada.

Ventajas sobre los MACs.

- **Públicamente verificables:** cualquiera con la clave pública verifica (un MAC necesita la clave secreta para verificar).

- **Transferibles:** se pueden reenviar a varios destinatarios o por una cadena y siguen siendo verificables en cada eslabón.
- **No repudio:** quien firma no puede negar haberlo hecho. Como la única forma de generar la firma es con la clave privada (y se asume que nunca se comparte), tiene valor legal análogo a la firma de puño y letra (en Argentina hay ley de firma digital; un organismo del Estado valida que el dueño de la clave privada es tal persona real).

Seguridad: $\text{Sig-forge}_{A,\Pi}$ (**falsificación**). Análoga a Mac-forge : (1) $(sk, pk) \leftarrow K$; (2) A obtiene $f(x) = \text{Sign}_{sk}(x)$ y pk ; (3) A hace las consultas de firma que quiera (conjunto Q); (4) A emite (m, s) con $m \notin Q$. $\text{Sig-forge}_{A,\Pi} = 1$ si $\text{Vrfy}_{pk}(m, s) = 1$ y $m \notin Q$. Si ningún A gana, la firma es **infalsificable**.

4.6.1. RSA-Signature y por qué se firma el HASH

Idéntico a RSA-Encryption pero **invirtiendo los roles de las claves**:

$$\text{Sign}_{sk}(m) \equiv m^d \text{ mód } n, \quad \text{Vrfy}_{pk}(m, s) : m = s^e \text{ mód } n.$$

△ Cuidado — error típico

RSA-Signature básico (sin hash) es INSEGURO. Ataques de falsificación:

- Elegir $s \leftarrow S$ al azar, calcular $m = s^e \text{ mód } n$, emitir (m, s) : **pasa la verificación**.
- **Maleabilidad:** con $s_1 = f(m_1)$, $s_2 = f(m_2)$, el par $(m_1 \cdot m_2, s_1 \cdot s_2)$ también verifica.

Hashed RSA (la firma que se usa). Solución elegante: en vez de firmar el mensaje, firmar su **hash** con una función de hash criptográfica (libre de colisiones):

$$\text{Sign}_{sk}(m) \equiv H(m)^d \text{ mód } n, \quad \text{Vrfy}_{pk}(m, s) : H(m) = s^e \text{ mód } n.$$

► En el parcial

Firma RSA en el parcial (2015):

- Firmar: $\sigma = H(m)^d \text{ mód } n$.
- Verificar (Vrfy): recibir m y σ , calcular $\sigma^e \text{ mód } n$ y **comparar con $H(m)$** .
- **Por qué se firma el HASH y no el mensaje:** (1) la resistencia a preimagen del hash evita el primer ataque (no se puede partir de s y construir un m válido); (2) no poder predecir $H(m)$ evita el ataque de maleabilidad; (3) bonus: el hash es de tamaño fijo, así la firma tiene tamaño fijo independiente del mensaje (RSA básico solo firma mensajes chicos del tamaño del bloque).

Hashed RSA no tiene prueba de seguridad salvo asumiendo un modelo ideal de H .

DSS (Digital Signature Standard). Algoritmo de firma basado en ElGamal (usa $H = \text{SHA1/SHA2}$). Genera claves (p, q, g, y) / (p, q, g, x) con q primo, p primo tal que $q \mid p - 1$, g generador de orden q , $x \leftarrow \mathbb{Z}_q$, $y = g^x \text{ mód } p$. Firma (r, s) con $r = (g^k \text{ mód } p) \text{ mód } q$ y $s = [H(m) + x \cdot r] \cdot k^{-1} \text{ mód } q$. Hereda las ventajas de ElGamal sobre RSA: existe sobre curvas elípticas (ECDSA) $\Rightarrow$ firmas más compactas con claves más chicas.

► En el parcial

Firma digital para distribución de software / binarios (2017): el proveedor firma el binario con su clave privada; cualquiera verifica con la clave pública que el binario es auténtico y no fue modificado. Aprovecha que la firma es *públicamente verificable* y da integridad/autenticación de origen sin compartir secretos.

△ Cuidado — error típico

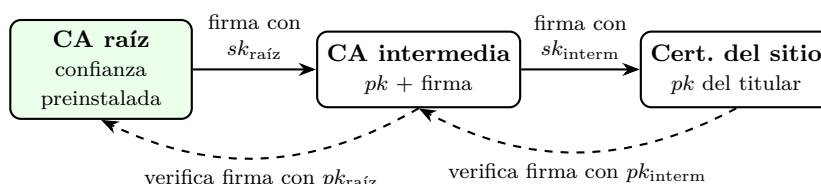
Diferencia entre cifrar y firmar (no confundir):

- **Cifrar** → confidencialidad. Se cifra con la **pública** del destinatario, descifra con su privada.
- **Firmar** → integridad / autenticación / no repudio (*no* da confidencialidad). Se firma con la **privada** del emisor, verifica cualquiera con la pública.

4.7. PKI y certificados digitales

El intercambio asimétrico resuelve la confidencialidad pero traslada el problema: hay que distribuir la clave pública **de forma autenticada** (saber que la pública de Bob es realmente de Bob, si no, un MiTM publica la suya). Esto lo resuelve la **PKI** (Public Key Infrastructure) con **certificados** y **Autoridades de Certificación (CA)**.

- Un **certificado digital** asocia una identidad (titular) con su **clave pública**, y está **firmado por la CA** (con la clave privada de la CA).
- **Cadena de confianza:** se confía en la CA (cuya clave pública se conoce de antemano); la CA firma el certificado del titular. Pueden encadenarse (CA raíz → CA intermedia → titular).



Cadena de certificados: cada nivel firma el certificado del siguiente con su clave privada. Validar = verificar las firmas hacia arriba hasta la CA raíz, en la que se confía de antemano.

► En el parcial

Certificados en el parcial:

- **Validar un certificado = verificar la FIRMA de la CA** (con la clave pública de la CA). (2018, 2017.)
- Un certificado contiene la clave **PÚBLICA** del titular, **NO** la clave privada de la CA. V/F “el certificado contiene la clave privada de la CA” → **FALSO**. (2025-1, 2025-2.)
- Propiedades clave pública/privada: la privada se mantiene secreta, la pública se distribuye de forma **autenticada** (justamente para eso sirve el certificado). (2015, 2025-1.)

△ Cuidado — error típico

Errores típicos de esta unidad:

- Pensar que el certificado contiene la clave *privada* (contiene la pública del titular, firmada

por la CA).

- Creer que **MD5 es asimétrico**: MD5 es una **función de hash**, no un esquema de clave pública.
- Confundir **DH (acuerdo de clave)** con **transporte de clave**: DH deriva una clave común, no envía una clave preexistente.

5. Protocolos

Hasta ahora teníamos primitivas sueltas: confidencialidad (cifrado), integridad (MAC/hash), cifrado autenticado, criptografía asimétrica y firma digital. Esta unidad las **compone** en *protocolos*: secuencias de mensajes que, sobre un canal inseguro, le dan a las partes confidencialidad, integridad y autenticación de quién está del otro lado. Cierra con un tema aparte —el **secreto compartido de Shamir**— que se toma mucho en el segundo parcial.

“Ahora lo que estamos haciendo es componer las piezas para poder hacer lo que queremos, que es transmitir información de forma que llegue solo a quien queremos, con garantía de que no fue alterada y de que somos verdaderamente la entidad que la emitió.”

— dicho en clase

5.1. Qué es un protocolo criptográfico

Un criptosistema CCA-Secure permite enviar $\text{Enc}_k(M)$ manteniendo **confidencialidad** e **integridad**, *pero requiere que ambas partes conozcan una misma clave k* . El problema central de los protocolos es justamente cómo llegar a esa clave compartida y cómo garantizar con quién se habla.

Protocolo de intercambio de claves. Es un protocolo $\Pi(n)$ ejecutado por dos partes:

- $\Pi : (n) \rightarrow \text{Tran}, k_a, k_b$.
- **No tiene entrada**, salvo el parámetro de seguridad n (que fija la longitud de las claves, es decir la seguridad computacional).
- La salida es: un conjunto de mensajes intercambiados (la *traza* o *log*), una clave k_a conocida solo por una de las partes y una clave k_b conocida solo por la otra.
- **Condición fundamental de correctitud:** $k_a = k_b$ (ambos llegan a la misma clave).

Tipos de ataque que se quieren evitar.

- **Ataques pasivos (snooping):** el atacante solo mira los mensajes e intenta deducir la clave o los datos.
- **Ataques activos:** el atacante juega un rol dentro del protocolo. Puede *omitir* mensajes, *re-escribir* su contenido, *reordenarlos* o *repetirlos*. Aparece el **Man-in-the-Middle (MitM)**: el atacante se para en el medio y le hace creer a cada lado que es la contraparte.

Definición: Nonce y frescura

Un **nonce** (number used once) es un número aleatorio de un solo uso. Aporta **frescura**: permite atar un mensaje a una ejecución concreta del protocolo y así detectar/evitar **replays** (repeticiones de mensajes viejos). El otro mecanismo de frescura es el **timestamp** (marca de tiempo).

5.2. Distribución de claves: KDC (simétrico) vs PKI (asimétrico)

El problema de la clave simétrica es *cómo distribuirla*. Hay dos grandes enfoques.

Esquema simétrico: KDC. Existe un tercero de confianza, el **KDC** (Key Distribution Center). *Hipótesis de arranque:* cada entidad **ya comparte una clave con el KDC** (se estableció físicamente / fuera de banda). El KDC genera claves de sesión entre pares. Es la base de **Kerberos** y de Active Directory (con modificaciones).

PGP (mencionado). Protocolo antiguo (Pretty Good Privacy, años '90) basado en *delegación* / *transferencia de confianza*: yo confío en alguien que vi físicamente, y le sirvo de “KDC” a un tercero, encadenando confianza (incluso con grados). Ya no se ve en la materia porque casi no se usa.

Esquema asimétrico: PKI. En el mundo de clave pública el equivalente del KDC es la **PKI** (Public Key Infrastructure). Resuelve el problema de asociar una **clave pública** con una **identidad**. El ataque que busca evitar es el MitM/*spoofing*: si A pide “la clave de B ” y un atacante E le responde con pk_e en vez de pk_b , A cree hablar con B pero en realidad cifra con la clave de E . **No es aplicable a criptosistemas simétricos.**

► En el parcial

KDC vs PKI (2016, 2018, 2023). Hay que poder explicar similitudes y diferencias:

- **Similitud:** ambos descansan en un **tercero de confianza** y un punto de anclaje (la clave compartida con el KDC; los certificados raíz en PKI).
- **KDC:** simétrico, *online* (el KDC participa activamente en cada intercambio), conoce/maneja las claves. Si el KDC se compromete, todo deja de tener sentido.
- **PKI:** asimétrico, las CA firman certificados que pueden validarse *offline*; la CA no necesita estar online en cada comunicación. La confianza forma un árbol/cadena hasta una raíz autofirmada.

5.2.1. Análisis de protocolos KDC: Needham-Schroeder paso a paso

Notación: $\{M\}_k = e_k(M)$ (todo lo que está entre llaves está cifrado con k). k_a = clave de A con el KDC, k_b = clave de B con el KDC, k_s = clave de sesión nueva.

Primera aproximación (insegura).

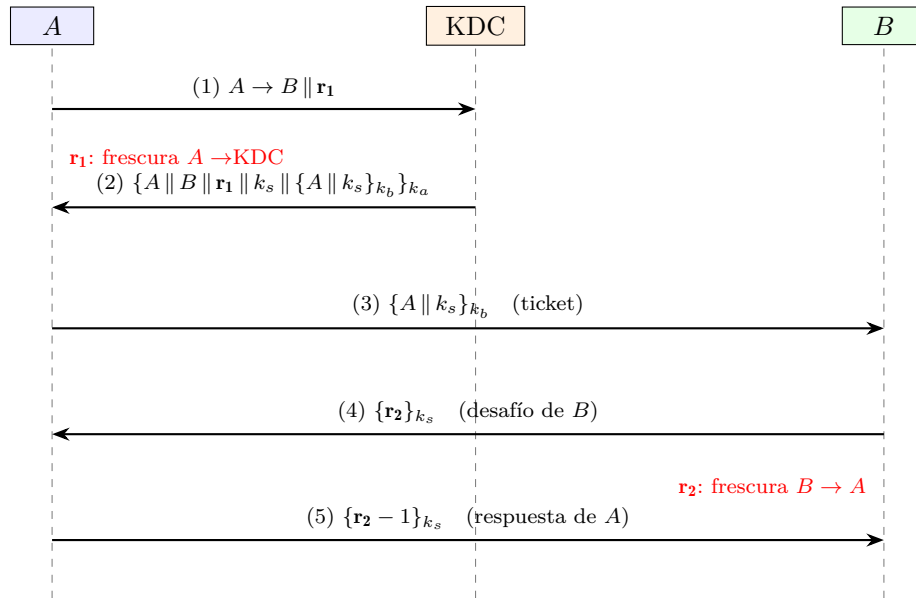
$$\begin{aligned} A &\rightarrow \text{KDC} : \quad \{\text{Sesión } A \rightarrow B\}_{k_a} \\ \text{KDC} &\rightarrow A : \quad \{k_s\}_{k_a} \parallel \{k_s\}_{k_b} \\ A &\rightarrow B : \quad \{k_s\}_{k_b} \end{aligned}$$

Problemas:

- **Replay:** un atacante graba $\{k_s\}_{k_b}$ y lo reenvía; B no tiene forma de saber que no está hablando con A .
- **Key reuse:** un atacante graba el mensaje del KDC y se lo reenvía a A cuando A inicia otra conversación; A y B terminan usando la misma clave de sesión vieja.

Segunda aproximación (con nonces).

$$\begin{aligned} A &\rightarrow \text{KDC} : \quad A \rightarrow B \parallel r_1 \\ \text{KDC} &\rightarrow A : \quad \{A \parallel B \parallel r_1 \parallel k_s \parallel \{A \parallel k_s\}_{k_b}\}_{k_a} \\ A &\rightarrow B : \quad \{A \parallel k_s\}_{k_b} \\ B &\rightarrow A : \quad \{r_2\}_{k_s} \\ A &\rightarrow B : \quad \{r_2 - 1\}_{k_s} \end{aligned}$$



Needham-Schroeder (2.^a aproximación). El KDC (T) genera la clave de sesión k_s ; los nonces r_1 (rojo, mensajes 1-2) y r_2 (rojo, mensajes 4-5) aportan frescura y evitan replays. El mensaje (3) es el ticket cifrado con k_b que solo B puede abrir.

Por qué funciona:

- **2.º mensaje:** cifrado con k_a (compartida A-KDC) $\Rightarrow$ proviene del KDC; trae el r_1 que A envió $\Rightarrow$ no es replay (frescura).
- **4.º mensaje:** solo B puede enviarlo (sabe k_s); avisa a A un intento de comunicación con un nonce fresco r_2 .
- **5.º mensaje:** A confirma; B sabe que no es repetición porque recibe $r_2 - 1$ (demuestra que A “está vivo” y leyó r_2).

△ Cuidado — error típico

El nombre del destinatario importa. Incluir B dentro del bloque cifrado por el KDC evita que ese ticket se reusen para hablar con otra entidad: ata la clave de sesión a un par concreto $A-B$.

El problema que queda (replay de clave de sesión vieja comprometida). Si el atacante E consigue una **clave de sesión antigua** k_s , arranca directo por el **tercer paso**:

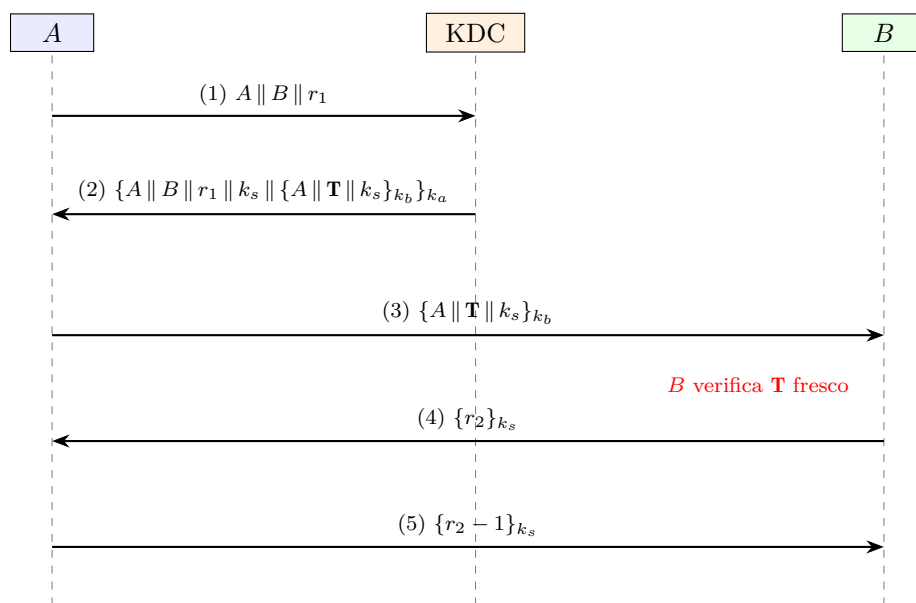
$$\begin{aligned}
 E \rightarrow B : & \quad \{A \parallel k_s\}_{k_b} \quad (\text{mensaje viejo grabado}) \\
 B \rightarrow E : & \quad \{r_2\}_{k_s} \\
 E \rightarrow B : & \quad \{r_2 - 1\}_{k_s}
 \end{aligned}$$

Como E conoce k_s , completa el challenge y **logra impersonar a A**. La causa: B no estuvo involucrado desde el principio, no tiene cómo verificar frescura del ticket.

Modificación Denning-Sacco: timestamps. El arreglo propuesto es agregar un **timestamp** T dentro del bloque destinado a B :

$$\begin{aligned}
 A &\rightarrow \text{KDC} : A \parallel B \parallel r_1 \\
 \text{KDC} &\rightarrow A : \{A \parallel B \parallel r_1 \parallel k_s \parallel \{A \parallel T \parallel k_s\}_{k_b}\}_{k_a} \\
 A &\rightarrow B : \{A \parallel T \parallel k_s\}_{k_b} \\
 B &\rightarrow A : \{r_2\}_{k_s} \\
 A &\rightarrow B : \{r_2 - 1\}_{k_s}
 \end{aligned}$$

Ahora B chequea que T no sea viejo $\Rightarrow$ no se puede reutilizar un ticket antiguo (rompe el replay/key-reuse). Esta variante con timestamp es la idea de **Kerberos**.



*Variante Denning-Sacco: se agrega un **timestamp** T (rojo) dentro del ticket destinado a B en los mensajes (2) y (3). Al recibir (3), B comprueba que T no sea viejo, lo que impide reutilizar un ticket/clave de sesión antiguo. Es la idea base de Kerberos.*

► En el parcial

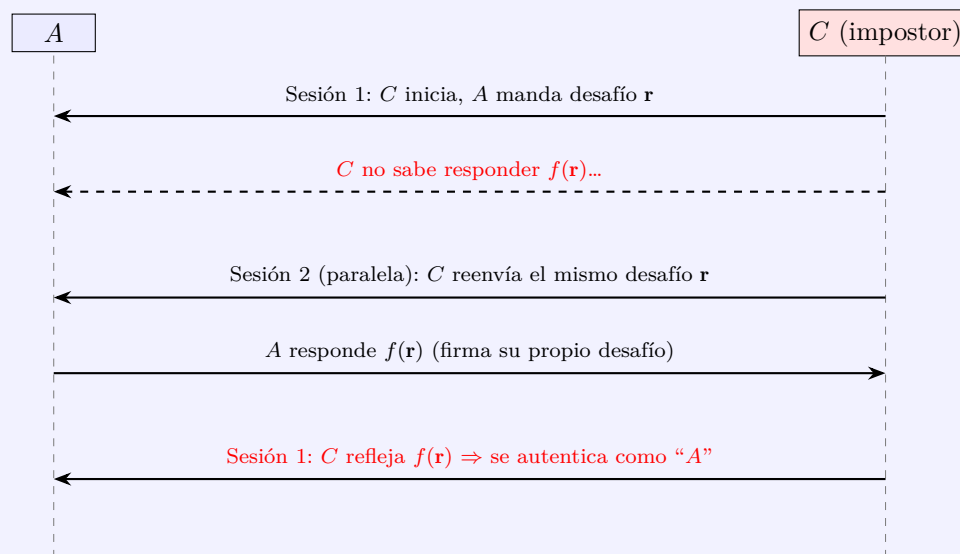
Ejercicio estrella: “analizar un protocolo” (Ej.1 reciente). Método para los handshakes tipo Needham-Schroeder (2018):

1. **Identificar el tipo:** intercambio de claves con KDC / tercero T .
2. Para cada mensaje preguntarse *quién lo pudo generar* (qué clave lo cifra) y *qué garantiza* (frescura por nonce/timestamp, identidad por el nombre incluido, anti-replay).
3. **Detectar el ataque:** típicamente *replay de una clave de sesión vieja comprometida* porque el destinatario no participó desde el inicio.
4. **Proponer el arreglo:** Denning-Sacco agrega **timestamps** para frescura/anti-replay (o atar la comunicación con B desde el primer mensaje).

► En el parcial

Ataque por reflexión (2015). En un protocolo de challenge-response con nonces (firma del desafío), C se autentica como A **abriendo una sesión paralela** y reusando el desafío

que recibió: le pide a *A* que le firme/responda su propio challenge y lo refleja en la sesión original. *Defensa*: romper la simetría de los mensajes (incluir identidades/dirección, nonces distintos por sentido).



Ataque por reflexión. *C* no puede responder el desafío r que le pide *A*; abre una **sesión paralela** (sesión 2), le reenvía el mismo r a *A* y obtiene $f(r)$, que luego refleja en la sesión original. *Defensa*: romper la simetría (incluir identidad/dirección, nonces distintos por sentido).

5.3. Certificados y PKI

Definición: Certificado

Mensaje que asocia una **clave pública** a una **identidad**. Contiene al menos: información de identidad (ej. nombre), la clave pública asociada, fecha de emisión, intervalo de validez y **tipo de uso** de la clave (firma de mensajes, cifrado de emails, firma de certificados, cifrado de sitios web). Está **firmado digitalmente por una autoridad competente (CA)**.

Cómo se autoprotege el certificado (integridad intrínseca). Se toma toda la información del certificado (incluida la clave pública del sujeto), se le calcula un **hash** y ese hash se firma con la **clave privada de la CA**; la firma se adosa al certificado. Si se altera *cualquier* bit (incluidos los bits de la firma), la firma deja de validar $\Rightarrow$ se detecta. Por eso el certificado puede distribuirse públicamente (lo puede dar el propio *B*).

“La clave pública se distribuye firmada con la clave privada de la autoridad certificante.”

— dicho en clase

Uso del certificado por *A*. *A* solicita el certificado de *B* (puede darlo *B* o cualquiera) y puede:

- **Verificar la identidad:** comparar el **CN** (Common Name) con lo que espera. Ej.: para sitios web, CN = dominio (**mipanaderia.com**); para email, CN = dirección; para servidores, CN = IP/hostname; para empresas, CN = razón social. El browser, al conectarse a **santander.com**, exige que el CN coincida con el hostname al que se conecta; si no, corta.
- **Obtener la clave pública** de *B* (está en el propio certificado, junto con el tipo: RSA-2048, DSA-EC 320, etc.).

- **Verificar validez:** tipo de uso permitido y fecha de vigencia.
- **Verificar integridad:** validar la firma digital de la CA (requiere la clave pública de la CA).

5.3.1. Cadenas de firmas y CA raíz

Las CA tienen a su vez un certificado, firmado por otra CA, y así hasta llegar a las **CA raíz**, que firman su propio certificado (**autofirmados**: issuer = subject) y son el **punto de confianza del sistema**. Se arma una **cadena de certificación**: se valida el certificado de *B* con la clave pública de quien lo firmó, subiendo recursivamente hasta una raíz confiable.

$$C_a = C'_a \parallel C_{CA3} \parallel C_{CA2} \parallel C_{CA1} \quad (\text{cert. de } A \text{ firmado por CA3})$$

$$C_b = C'_b \parallel C_{CA1} \quad (\text{cert. de } B \text{ firmado por CA1})$$

No existe una única CA raíz. Hay un conglomerado (gobiernos, grandes tecnológicas, empresas comerciales tipo Verisign/Thawte). Si *A* y *B* tienen CA distintas: o bien *A* confía en la CA de *B*, o bien las CA se **certifican entre sí** (certificación cruzada). En la práctica existe una **lista de CA reconocidas** en el sistema operativo, los navegadores y runtimes (JVM).

! Importante — remarcado en clase

Cómo se distribuyen las raíces (pregunta tomada en clase). No por un servidor online (eso permitiría un MitM en el punto de anclaje). Vienen **hardcodeadas en el binario**: en la base de certificados raíz del sistema operativo y de los browsers, que se actualizan al actualizar el binario. Si Chrome muestra “no confiable” ante un certificado autofirmado, es porque no puede trazar una cadena hasta una CA en la que ya confía.

Revocación. Hay que poder invalidar una clave **antes** de su expiración (fue averiguada por un atacante, o cambio de dueño). En X.509 **solo el emisor** puede revocar un certificado.

- **CRL (Certificate Revocation List):** lista de certificados revocados (análoga a la lista de tarjetas robadas). Listado actual (vigentes y revocados) o histórico (expirados y revocados). Se puede validar offline u online.
- **OCSP (Online Certificate Status Protocol):** servicio online con API estandarizada, administrado por las CA, que certifica que una firma está “activa”. El resultado es un certificado firmado, de corta duración (1 hora a 7 días). **OCSP Stapling:** se incluye en la cadena para que el cliente no tenga que consultar una fuente externa (recomendado para sitios de alto tráfico).

► En el parcial

Cadena de validaciones de un certificado (X.509), se toma casi siempre. Dado un servidor (p. ej. www.itba.edu.ar con cadena Server → Root → Chain), describir **todas** las validaciones:

1. Obtener la clave pública del emisor (en la cadena, o del sistema si es raíz) y verificar **recursivamente** el certificado del emisor hasta una raíz confiable (verificar el *cert path*).
2. Verificar **integridad**: validar la firma con el algoritmo especificado y la clave pública del emisor.
3. Verificar **intervalo de validez**: el certificado debe estar vigente, y la CA debe estar vigente al comienzo del período de vigencia. (Errores típicos en sistemas embebidos sin reloj/fecha actualizada.)

4. Verificar **identidad**: el CN coincide con lo que espera la aplicación.
5. Verificar **uso** permitido del certificado.
6. Chequear que no esté en **listas de revocación** (CRL/OCSP).

5.4. Canal seguro: SSL / TLS

Provee **confidencialidad + integridad + autenticación** de los participantes (origen y destino) sobre un canal inseguro, a **nivel de transporte** (típicamente sobre TCP). Es transparente desde la API: como un socket donde se meten y sacan datos.

- **SSL** (Secure Socket Layer): versión inicial de Netscape.
- **TLS** (Transport Layer Security): evolución de SSL (TLS 1.2 = SSL 3.3), estándar actual. Es la base de **HTTPS** y **depende de la PKI**.

TLS Record. TLS recibe el mensaje, lo divide en bloques de no más de 2^{16} bytes, comprime cada bloque y calcula su hash, **cifra** cada bloque y su hash, y lo envía al servicio inferior (TCP). Nota: es el esquema seguro *encrypt-then-MAC* (primero cifrar, después calcular el MAC sobre lo cifrado).

Tipos de mensaje TLS.

- **Handshake**: configuración inicial, autenticación y negociación de material criptográfico.
- **Application data**: información del protocolo encapsulado.
- **Alert**: eventos/problemas (fuera de banda); pueden ser warning o fatal (un fatal invalida la conexión); `CloseNotify` marca fin de sesión.
- **Change Cipher Spec**: concluye una renegociación de claves; no tiene contenido, es un tipo de mensaje; lo puede pedir cliente o servidor en cualquier momento.

Criptografía negociable.

- **Intercambio de clave**: RSA; Diffie-Hellman con certificado RSA o DSS; DH anónimo; ECDH (DH sobre curvas elípticas); Kerberos. (Hay variantes obsoletas de 512 bits por restricciones de exportación de EE.UU.)
- **Cifrado simétrico**: ChaCha20, 3DES-EDE-CBC, AES (CBC/CCM/**GCM**, recomendado), IDEA-CBC, ARIA. (RC4 y DES-CBC obsoletos.)
- **Hash/MAC**: HMAC-SHA2 (256/384), AEAD. (HMAC-MD5/SHA1 desaconsejados.)
- **Autenticación**: firmas RSA o DSS.

Sesión vs conexión. La **sesión** es una asociación entre dos pares (soporta múltiples conexiones): tiene ID de sesión, certificado X.509v3 opcional del otro extremo, método de compresión y de cifrado/MAC, y el **Master Secret** (clave de sesión compartida de 48 bytes). La **conexión** describe cómo intercambiar datos: bits aleatorios de calidad criptográfica, claves de escritura distintas para cada lado, claves para los MAC, IVs y números de secuencia para cliente y servidor.

5.4.1. TLS Handshake

Parte 1 – Hello.

$$\begin{aligned} C \rightarrow S : & \quad \{ V_c \parallel r_1 \parallel S_{id} \parallel \text{Ciphers} \parallel \text{Comps} \} \quad (\text{ClientHello}) \\ S \rightarrow C : & \quad \{ V \parallel r_2 \parallel S_{id} \parallel \text{Cipher} \parallel \text{Comp} \} \quad (\text{ServerHello}) \end{aligned}$$

V_c = versión del cliente; V = $\min(V_c, V_s)$; r_1, r_2 = nonces (timestamp + 28 bytes aleatorios); $S_{id} = 0$ inicia una nueva sesión; Ciphers = lista de stacks disponibles, Cipher = elegido; ídem: Comps/Comp (compresión).

Parte 2 – Certificado del servidor.

$$\begin{aligned} S \rightarrow C : & \quad \{ \text{Certificado} \} \quad (\text{si el servidor se autentica}) \\ S \rightarrow C : & \quad \{ p \parallel \{ \text{hash}(r_1 \parallel r_2 \parallel p) \}_{k_s} \} \quad (\text{ServerKeyExchange}) \\ S \rightarrow C : & \quad \{ \text{ctype} \parallel \text{CAs} \} \quad (\text{CertificateRequest, si el cliente se autentica}) \\ S \rightarrow C : & \quad \{ \} \quad (\text{ServerHelloDone}) \end{aligned}$$

r_1, r_2 son los nonces de la parte inicial (evitan replay). p = parámetros criptográficos (e, n para RSA; p, g, g^a para DH). k_s = clave privada del servidor correspondiente a la pública del certificado: al firmar el hash con k_s , el cliente puede validar con la clave pública del certificado.

Parte 3 – ClientKeyExchange.

$$\begin{aligned} C \rightarrow S : & \quad \{ \text{Certificado} \} \quad (\text{Client Certificate, solo si fue solicitado}) \\ C \rightarrow S : & \quad \{ V \parallel \text{PRE}_{rand} \}_{k_s} \quad (\text{versión RSA}) \\ C \rightarrow S : & \quad \{ g^b \text{ mód } p \}_{k_s} \quad (\text{versión DH, con } \text{PRE} = g^{ab} \text{ mód } p) \end{aligned}$$

V = versión informada originalmente por el cliente (previene **downgrade attacks**); g, p = parámetros de DH informados por el servidor.

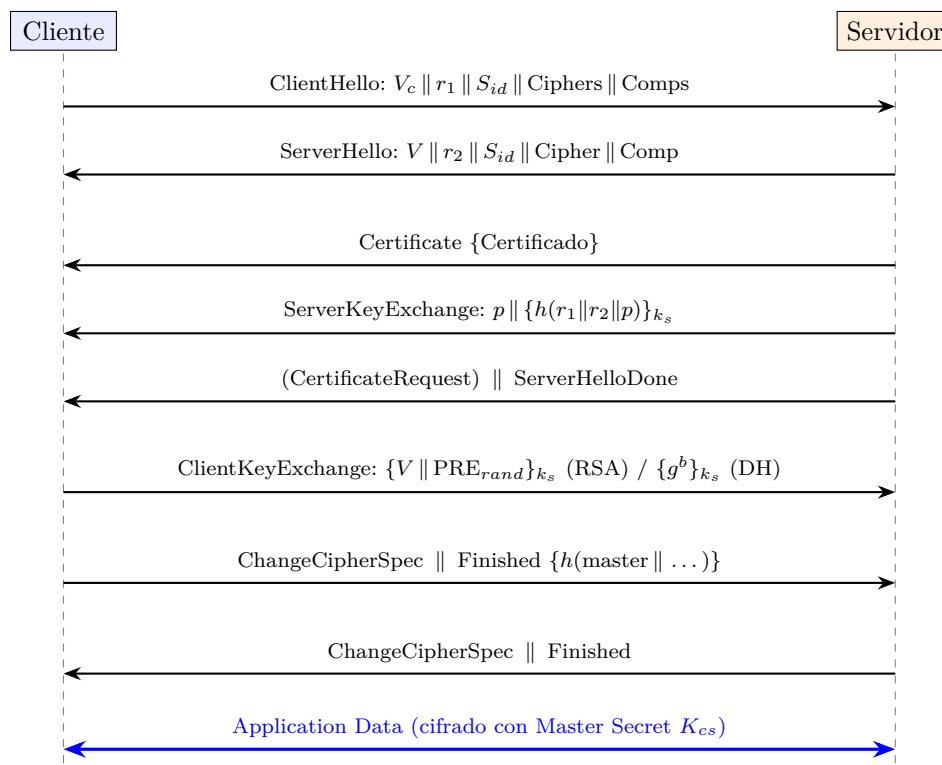
Parte 4 – Change Cipher Spec + Finished.

$$\begin{aligned} C \rightarrow S : & \quad \{ \} \quad (\text{ChangeCipherSpec; desde aquí el cliente usa los parámetros negociados}) \\ C \rightarrow S : & \quad \{ h(\text{master} \parallel \text{opad} \parallel h(\text{msgs} \parallel \text{master} \parallel \text{ipad})) \} \quad (\text{Finished}) \end{aligned}$$

y simétricamente el servidor envía su ChangeCipherSpec y su Finished. El **Master Secret** se deriva como

$$\text{Master} = \text{MD5}(\text{pre} \parallel \text{SHA}(\text{'A'} \parallel \text{pre} \parallel r_1 \parallel r_2)) \parallel \text{MD5}(\text{pre} \parallel \text{SHA}(\text{'BB'} \dots)) \parallel \text{MD5}(\text{pre} \parallel \text{SHA}(\text{'CCC'} \dots))$$

donde **msgs** = concatenación de todos los mensajes intercambiados hasta el momento.



*Handshake TLS/SSL. Hello (negociación de versión, nonces r_1, r_2 , cipher y compresión), certificado y ServerKeyExchange (autenticación del servidor), ClientKeyExchange (acuerdo de la clave), y ChangeCipherSpec + Finished en ambos sentidos (los Finished atan todo el handshake con un hash de *msgs*). Recién entonces fluyen los datos cifrados con el Master Secret.*

► En el parcial

Handshake tipo TLS/SSL (2023). Puntos que se piden:

- **Identificarlo como handshake** (negociación + autenticación + acuerdo de claves).
- Los mensajes **Finished** confirman las claves e **integridad de todo el handshake** (incluyen un hash de *todos* los mensajes previos, atando la conversación y evitando manipulaciones intermedias / downgrade).
- Se deriva una **clave de sesión** K_{cs} (el Master Secret) en vez de usar directamente K_0 /la clave del certificado: esto es **separación de claves** (la clave de larga vida no se usa para cifrar datos).

Panorama / ataque a TLS. Es lengua franca de la web (un sitio sin TLS se considera inseguro). El MitM “a la fuerza” requiere muchas condiciones: hacer **DNS spoofing** (redirigir **santander.com** a una IP propia), presentar un **certificado autofirmado/trucho** a nombre del sitio, y lograr que el cliente lo acepte (que su lista de CA confíe en él, o que el usuario ignore el warning, o que el cliente **no chequee bien el cert path** —bug de programación real y recurrente, p. ej. trust stores mal configurados pensados para desarrollo que se filtran a producción). Por eso el ataque más viable termina siendo contra el *usuario* (phishing), no contra el protocolo.

5.5. Intercambio de claves: Diffie-Hellman y Shamir’s no-key

Diffie-Hellman (en contexto de protocolo). Acuerdo de clave cuya seguridad descansa en el **problema del logaritmo discreto (DLP)**. En TLS se usa con certificados (RSA/DSS) o sobre curvas elípticas (ECDH).

► En el parcial

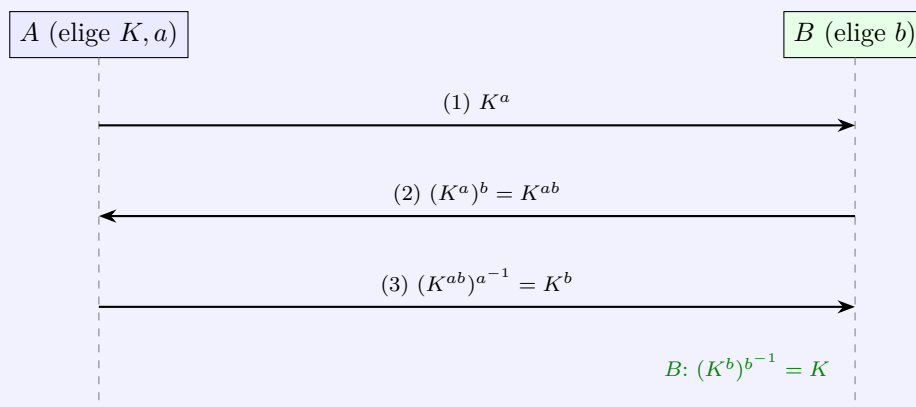
Diffie-Hellman puro (2025-1). Hay que: identificarlo, dar un ejemplo numérico, indicar los **valores válidos de q** (el primo/parámetros del grupo) y justificar la seguridad en el **DLP**. **Problema central:** DH puro *no tiene autenticación* $\Rightarrow$ es susceptible a **MitM** (por eso en TLS va con certificados). El profesor lo remarca: “DH va por un tubo, no hay chance de chequear nada” si alguien se para en el medio.

► En el parcial

Shamir’s no-key protocol (2017). (*Ojo: es distinto del secreto compartido de Shamir.*) Intercambio de 3 pasos **sin clave previa** para transportar una clave K elegida por A :

1. $A \rightarrow B$: K^a (A elige exponente a).
2. $B \rightarrow A$: $(K^a)^b = K^{ab}$ (B aplica su exponente b).
3. $A \rightarrow B$: $(K^{ab})^{a^{-1}} = K^b$ (A “quita” su a).
4. B aplica b^{-1} : $(K^b)^{b^{-1}} = K$.

La seguridad está en el **DLP**. Se diferencia de DH en que **no acuerda una clave nueva**, sino que **transporta una clave elegida por A** .



Shamir’s no-key protocol (3 pasos, sin clave previa). A “envuelve” K con su exponente a ; B agrega b ; A quita a con a^{-1} ; finalmente B quita b con b^{-1} y obtiene K . La seguridad descansa en el DLP. Transporta una clave elegida por A (no la acuerda como DH).

5.6. Secreto compartido de Shamir (umbrales / threshold)

“Shamir, de la S de RSA, es un investigador israelí; armó este método basado en umbrales.”
— dicho en clase

Es de los ejercicios más recurrentes (se toma mucho en el **segundo parcial**). La idea: repartir un secreto en n **sombras** de modo que hagan falta t (**o k**) para reconstruirlo, y que con **menos de t** no se obtenga **ninguna** información.

Criptosistema de umbrales (terna de algoritmos).

Gen : $() \rightarrow K^N$	(genera N sombras)
Enc : $K^T \times P \rightarrow C$	(cifrado, usa T claves)
Dec : $K^T \times C \rightarrow P$	(descifrado, usa T claves)

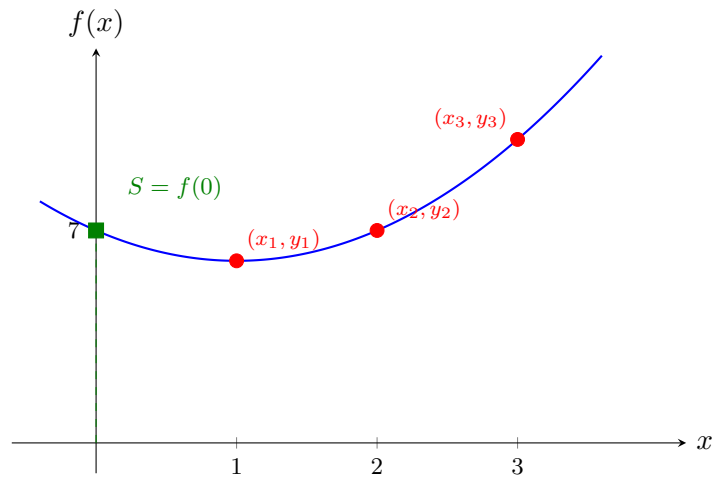
N = sombras disponibles; T = umbral (claves necesarias). Analogía: una caja fuerte que se cierra con 4 llaves pero basta con 3 cualesquiera para abrirla. Se usa en esquemas de votación y de clave compartida entre varios actores.

Definición: Esquema umbral (t, n)

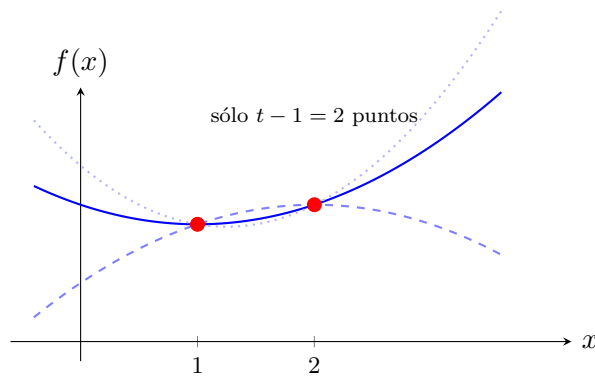
Se reparten n sombras; con t **cualquiera** se reconstruye el secreto; con **menos de t** no se obtiene ninguna información. Está demostrado que, faltando una sombra, averiguarla equivale al **secreto perfecto** (seguridad absoluta tipo Shannon).

5.6.1. Método (paso a paso)

Principio. Un polinomio de grado $t - 1$ queda determinado por su evaluación en t puntos distintos.



Secreto compartido de Shamir con $t = 3$: el secreto S se esconde como el término independiente $f(0)$ de un polinomio de grado $t - 1 = 2$ (una parábola). Cada sombra es un punto (x_i, y_i) sobre la curva (rojo). Con t puntos cualesquiera la curva queda determinada y se recupera $S = f(0)$ (cuadrado verde sobre el eje y).



*Con **menos de t** sombras (aquí $t - 1 = 2$ puntos) hay **infinitas** curvas de grado $t - 1$ que pasan por ellas: el secreto $f(0)$ queda totalmente indeterminado (entropía máxima, seguridad perfecta tipo Shannon).*

Reparto (cifrado). Sea S el secreto, en $\mathbb{Z}_p$ con p primo, $p > n$ y $p > S$.

- Tomar coeficientes aleatorios $a_1, \dots, a_{t-1} \in \{0, \dots, p-1\}$ y fijar $a_0 = S$.
- Armar el polinomio de grado $t - 1$:

$$f(x) = S + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \quad (\text{mód } p)$$

El término independiente $f(0) = S$ es el secreto.

- La **sombra** del participante i es el par $(i, f(i) \text{ mód } p)$, para $i = 1, \dots, n$.

Reconstrucción (descifrado) por interpolación de Lagrange en $\mathbb{Z}_p$. Con t sombras cualesquiera (x_j, y_j) :

$$S = f(0) = \sum_j y_j L_j(0) \pmod{p}, \quad L_j(0) = \prod_{m \neq j} \frac{x_m}{x_m - x_j} \pmod{p}$$

El **inverso modular** de $(x_m - x_j)$ se calcula con Euclides extendido o por Fermat ($a^{-1} \equiv a^{p-2} \pmod{p}$).

△ Cuidado — error típico

Error típico: fallar en la **aritmética modular** —no reducir mod p , o invertir mal—. Cada división es una multiplicación por el inverso modular; no se puede “dividir” directamente. Y en la parte (c) (ver abajo), no expresar el umbral en términos de **entropía**.

5.6.2. Ejemplo completo resuelto $(t, n) = (3, 5)$ en $\mathbb{Z}_{11}$

Cifrado. Secreto $S = 7$, esquema $(3, 5) \Rightarrow$ polinomio de grado $t - 1 = 2$:

$$f(x) = 5x^2 + 3x + 7 \pmod{11}$$

Sombras (evaluando en $x = 1, \dots, 5$):

$$\begin{aligned} f(1) &= 5 + 3 + 7 = 15 \equiv 4 \pmod{11} \\ f(2) &= 20 + 6 + 7 = 33 \equiv 0 \pmod{11} \\ f(3) &= 45 + 9 + 7 = 61 \equiv 6 \pmod{11} \\ f(4) &= 80 + 12 + 7 = 99 \equiv 2 \pmod{11} \\ f(5) &= 125 + 15 + 7 = 147 \equiv 4 \pmod{11} \end{aligned}$$

Sombras: $(1, 4)$, $(2, 0)$, $(3, 6)$, $(4, 2)$, $(5, 4)$.

Descifrado tomando tres cualesquiera, p. ej. $(2, 0)$, $(3, 6)$, $(5, 4)$:

$$\begin{aligned} f(x) &= 0 \cdot \frac{(x-3)(x-5)}{(2-3)(2-5)} + 6 \cdot \frac{(x-2)(x-5)}{(3-2)(3-5)} + 4 \cdot \frac{(x-2)(x-3)}{(5-2)(5-3)} \\ &= \left[0 - 3(x^2 - 7x + 10) + \frac{4}{6}(x^2 - 5x + 6) \right] \pmod{11} \\ &= 5x^2 + 3x + 7 \end{aligned}$$

y el secreto es $s = f(0) = 7$. ✓

► En el parcial

Ejemplo real adicional para practicar. Esquema $(k, n) = (3, 4)$ en $\mathbb{Z}_7$, con $C = \{(1, 6), (2, 1), (3, 0), (4, 3)\}$. Recuperar el polinomio y el secreto: el resultado esperado es

$$f(x) = 1 + 3x + 2x^2 \pmod{7}, \quad S = f(0) = 1.$$

Verificación rápida: $f(1) = 1 + 3 + 2 = 6$; $f(2) = 1 + 6 + 8 = 15 \equiv 1$; $f(3) = 1 + 9 + 18 = 28 \equiv 0$; $f(4) = 1 + 12 + 32 = 45 \equiv 3 \pmod{7}$. ✓

► En el parcial

Parte (c) típica: el umbral en términos de entropía.

- Con **menos de t sombras**, la entropía del secreto $H(S)$ se mantiene **máxima**: no se filtra

ninguna información (todos los valores del secreto siguen siendo igualmente probables)
 $\Rightarrow$ seguridad perfecta tipo Shannon.

- Con t **sombras**, $H(S) = 0$: el secreto queda completamente determinado.

! Importante — remarcado en clase

Shamir funciona sobre **cualquier cuerpo finito**, no solo $\mathbb{Z}_p$: también sobre cuerpos de extensión $\mathbb{F}_{2^n}$.